

Pemodelan Komputasi Sistem Kamera

Termal Sensor CMOS, Elektromagnetik Lensa, Optika Gelombang, dan Penyerapan
Foton Kuantum

Kelompok 5

Sandy Fauzi Amrulloh (140310240054) Adinda Yulia Putri Rachmat (140310240052)
Aisyah Nur Khairan (140310240014)

Program Studi Fisika, FMIPA, Universitas Padjadjaran

Juni 2026

Abstrak

Laporan ini menyajikan pemodelan komputasi untuk tujuh subsistem fisis pada kamera *mirrorless* dan *action camera* digital yang diatur oleh berbagai Persamaan Diferensial Parsial (PDP). Subsistem yang dianalisis mencakup konduksi panas (metode FTCS 3D), elektrostatika silindris dan bola (metode FDFD dan *Finite Volume*), magnetostatika (Potensial Vektor), perambatan gelombang Maxwell (metode FDTD Yee 3D), serta persamaan Schrödinger bergantung waktu (metode *Crank-Nicolson*). Simulasi pada Kasus 1 dan 6 diselesaikan sepenuhnya dalam matriks ruang tiga dimensi (3D), sedangkan lima kasus lainnya dikomputasi menggunakan pendekatan simetri 2D (silindris dan azimuthal) untuk mengoptimalkan kinerja numerik tanpa mereduksi akurasi dari fenomena 3D aslinya. Setiap kasus memuat penurunan persamaan diskret, metodologi penyelesaian numerik yang diakselerasi melalui vektorisasi *NumPy* dan kompilasi *Numba*, serta analisis makna fisis dari hasil iterasi. Seluruh program menghasilkan *Output* berupa animasi dan *plot* statis pada *file Jupyter Notebook*, termasuk visualisasi proyeksi permukaan 3D (*3D surface*) untuk kasus bervolume spasial penuh. Kode sumber program Python disertakan secara lengkap pada lampiran.

Daftar Isi

1	Pendahuluan Umum	1
1.1	Latar Belakang	1
1.2	Rumusan Masalah	1
1.3	Tujuan	2
2	Kasus 1: Distribusi Panas Sensor CMOS dengan Metode FTCS Tiga Dimensi	2
2.1	Pendahuluan	2
2.1.1	Latar Belakang	2
2.1.2	Rumusan Masalah	3
2.1.3	Tujuan	3
2.2	Teori Dasar	3
2.2.1	Sensor CMOS dan Fenomena Pembangkitan Panas	3
2.2.2	Persamaan Konduksi Panas Tiga Dimensi	3
2.3	Metodologi	4
2.3.1	Diskritisasi Domain dan Skema FTCS	4
2.3.2	Algoritma dan Komputasi Program	5
2.4	Hasil dan Pembahasan	5
3	Kasus 2: Distribusi Potensial dan Medan Listrik Kapasitor Tabung Tegangan Tinggi pada Sistem <i>Flash</i> (Laplace Silindris)	6
3.1	Pendahuluan	6
3.1.1	Latar Belakang	6
3.1.2	Rumusan Masalah	7
3.1.3	Tujuan	7
3.2	Teori Dasar	7
3.2.1	Elektrostatika dan Potensial Skalar	7
3.2.2	Persamaan Laplace Koordinat Silinder	8
3.2.3	Solusi Koaksial Analitik sebagai Acuan	8
3.3	Metodologi	10
3.3.1	Geometri dan Diskritisasi Domain	10
3.3.2	Stensil Beda Hingga Konservatif	10
3.3.3	Algoritma dan Akselerasi Komputasi	10
3.4	Hasil dan Pembahasan	10
4	Kasus 3: Distribusi Medan Listrik pada Tetesan Kristal Cair (PDLC) <i>ND Filter</i> Elektronik dalam Medan Seragam (Volume Hingga)	12

4.1	Pendahuluan	12
4.1.1	Latar Belakang	12
4.1.2	Rumusan Masalah	12
4.1.3	Tujuan	12
4.2	Teori Dasar	13
4.2.1	Elektrostatika Medium Dielektrik dan Polarisasi	13
4.2.2	Persamaan Laplace Koordinat Bola	13
4.2.3	Solusi Klasik Bola Dielektrik	14
4.3	Metodologi	14
4.3.1	Diskritisasi Volume Hingga (FVM)	14
4.3.2	Algoritma dan Akselerasi Komputasi	14
4.4	Hasil dan Pembahasan	15
5	Kasus 4: Distribusi Medan Magnet Aktuator Voice Coil Motor (Magnetostatik Silindris)	16
5.1	Pendahuluan	16
5.1.1	Latar Belakang	16
5.1.2	Rumusan Masalah	16
5.1.3	Tujuan	17
5.2	Teori Dasar	17
5.2.1	Magnetostatika dan Potensial Vektor Magnetik	17
5.2.2	Persamaan Poisson pada Koordinat Silinder	18
5.2.3	Perhitungan Medan Magnet $\mathbf{B}$	18
5.2.4	Solusi Klasik Solenoida sebagai Acuan	18
5.3	Metodologi	18
5.3.1	Diskritisasi Volume Hingga (FVM)	18
5.3.2	Algoritma dan Akselerasi Komputasi	19
5.4	Hasil dan Pembahasan	19
6	Kasus 5: Distribusi Medan Magnet Magnet Permanen OIS sebagai Bola Termagnetisasi Seragam	20
6.1	Pendahuluan	20
6.1.1	Latar Belakang	20
6.1.2	Rumusan Masalah	21
6.1.3	Tujuan	21
6.2	Teori Dasar	21
6.2.1	Magnet Permanen dan Magnetisasi	21
6.2.2	Bola Termagnetisasi Seragam	21

6.3	Metodologi	22
6.3.1	Domain Simulasi Ruang Kosong 2D	22
6.3.2	Algoritma dan Akselerasi Komputasi	22
6.4	Hasil dan Pembahasan	23
7	Kasus 6: Propagasi Cahaya melalui <i>Dome Port</i> dan Lensa menuju Sensor CMOS (Yee <i>Leapfrog</i> FDTD)	23
7.1	Pendahuluan	23
7.1.1	Latar Belakang	24
7.1.2	Rumusan Masalah	24
7.1.3	Tujuan	24
7.2	Teori Dasar	24
7.2.1	Elektrodinamika dan Persamaan Maxwell	24
7.2.2	Skema Yee- <i>Leapfrog</i> FDTD (Finite-Difference Time-Domain)	25
7.3	Metodologi	25
7.3.1	Geometri Optik FDTD 3D	25
7.3.2	Algoritma Akselerasi Paralel Numba	26
7.4	Hasil dan Pembahasan	26
8	Kasus 7: Penyerapan Foton pada Fotodiode CMOS dengan <i>Crank-Nicolson</i> Dua Dimensi	27
8.1	Pendahuluan	27
8.1.1	Latar Belakang	27
8.1.2	Rumusan Masalah	28
8.1.3	Tujuan	28
8.2	Teori Dasar	28
8.2.1	Persamaan Schrödinger Bergantung Waktu (TDSE)	28
8.2.2	Kegagalan Euler dan Solusi dari Skema <i>Crank-Nicolson</i>	29
8.3	Metodologi	29
8.3.1	Konstruksi Hamiltonian dan CAP	29
8.3.2	Akselerasi Matriks Numba dan Faktorisasi LU	29
8.4	Hasil dan Pembahasan	30

Daftar Pustaka

Lampiran: Listing Program

Listing Program Kasus 1	
Listing Program Kasus 2	
Listing Program Kasus 3	

Listing Program Kasus 4	
Listing Program Kasus 5	
Listing Program Kasus 6	
Listing Program Kasus 7	

1 Pendahuluan Umum

1.1 Latar Belakang

Kamera *mirrorless* dan *action camera* digital merupakan integrasi rapat dari banyak subsistem fisis yang bekerja secara bersamaan. Cahaya dikumpulkan melalui jendela pelindung (*dome port*) dan susunan lensa, difokuskan oleh aktuator, lalu diubah menjadi sinyal listrik oleh *image sensor* yang sekaligus harus mengelola panas buangnya. Setiap subsistem diatur oleh persamaan diferensial parsial yang berbeda, mulai dari konduksi panas, elektrostatika, magnetostatika, persamaan gelombang Maxwell, hingga persamaan Schrödinger.

Dalam memodelkan kompleksitas sistem ini, laporan ini menyajikan analisis komputasi numerik secara terpadu melalui tujuh kasus simulasi. Setiap kasus dirancang untuk merepresentasikan satu fenomena fisis pada subsistem kamera, mulai dari penyebaran termal pada sensor gambar, perilaku medan listrik dan magnet pada komponen lensa, rambatan gelombang elektromagnetik, hingga penangkapan elektron foto-eksitasi secara kuantum. Melalui diskretisasi Persamaan Diferensial Parsial (PDP) dengan metode numerik seperti beda hingga (*Finite Difference*), volume hingga (*Finite Volume*), dan *Crank-Nicolson*, kajian ini diharapkan mampu memberikan pemahaman saintifik mengenai mekanisme fisis perangkat optoelektronik modern secara kuantitatif.

1.2 Rumusan Masalah

Secara umum, rumusan masalah laporan ini merupakan gabungan rumusan masalah ketujuh kasus berikut, yang diuraikan lebih rinci pada bagian masing-masing.

1. Bagaimana distribusi temperatur dan lokasi *hotspot* pada sensor CMOS selama perekaman *video* 4K (Kasus 1)?
2. Bagaimana distribusi potensial dan medan listrik pada kapasitor tabung tegangan tinggi sistem *flash*, termasuk efek tepi yang memicu risiko dadal listrik (Kasus 2)?
3. Bagaimana tetesan kristal cair (PDLC) pada *ND filter* elektronik menata ulang medan listrik penggerak seragam di sekitarnya (Kasus 3)?
4. Bagaimana distribusi medan magnet kumparan aktuator VCM sepanjang langkah autofokus (Kasus 4)?
5. Bagaimana distribusi medan magnet magnet permanen OIS yang dimodelkan sebagai bola termagnetisasi seragam (Kasus 5)?
6. Bagaimana cahaya merambat menembus *dome port* dan lensa lalu membentuk citra pada sensor CMOS (Kasus 6)?
7. Bagaimana elektron foto-eksitasi bergerak dan tertangkap di sumur deplesi fotodioda CMOS (Kasus 7)?

1.3 Tujuan

Sejalan dengan rumusan masalah di atas, tujuan laporan ini merupakan gabungan tujuan ketujuh kasus berikut, yang diuraikan lebih rinci pada bagian masing-masing.

1. Memetakan penyebaran panas dan kenaikan suhu sensor CMOS serta menilai risiko panas berlebih (Kasus 1).
2. Menyelesaikan persamaan Laplace silindris kapasitor tabung *flash* dan memvalidasinya terhadap solusi koaksial analitik (Kasus 2).
3. Menghitung medan listrik tereduksi di dalam tetesan kristal cair PDLC dan memvalidasinya terhadap teori bola dielektrik (Kasus 3).
4. Menyelesaikan persamaan potensial vektor magnetik dan memetakan medan magnet aktuator VCM (Kasus 4).
5. Menghitung medan interior seragam dan medan dipol eksterior magnet permanen OIS (Kasus 5).
6. Mensimulasikan propagasi gelombang Maxwell dan pembentukan citra pada sensor CMOS (Kasus 6).
7. Mensimulasikan dinamika kuantum elektron dan menganalisis efisiensi penangkapan muatan (Kasus 7).

2 Kasus 1: Distribusi Panas Sensor CMOS dengan Metode FTCS Tiga Dimensi

2.1 Pendahuluan

2.1.1 Latar Belakang

Perkembangan *action camera* telah memungkinkan perekaman *video* beresolusi tinggi hingga 4K dengan *frame rate* mencapai 60 *fps* atau lebih. Kemampuan ini meningkatkan kualitas visual yang dihasilkan, namun juga menyebabkan beban kerja elektronik pada sensor CMOS (*Complementary Metal-Oxide-Semiconductor*) dan rangkaian pendukungnya menjadi semakin besar. Selama proses perekaman, sensor melakukan pembacaan dan pemrosesan data secara kontinu sehingga sebagian energi listrik terdisipasi menjadi panas. Karena *action camera* umumnya memiliki dimensi yang ringkas, desain kedap air, dan sistem pendinginan pasif, pelepasan panas ke lingkungan menjadi terbatas. Akibatnya, panas cenderung terakumulasi pada sensor CMOS dan menyebabkan kenaikan temperatur. Temperatur yang tinggi dapat meningkatkan *thermal noise*, memunculkan *hot pixels*, menurunkan kualitas citra, serta memicu *thermal throttling* atau *overheating shutdown* pada perangkat. Distribusi temperatur pada sensor CMOS dapat dimodelkan menggunakan persamaan konduksi panas tiga dimensi dengan sumber panas internal. Karena solusi analitik sulit diperoleh untuk kasus ini, digunakan metode beda hingga

(*Finite Difference Method*/FDM) dengan skema *Forward-Time Central-Space* (FTCS) 3D. Melalui simulasi numerik, distribusi temperatur, lokasi *hotspot*, dan perkembangan suhu selama perekaman *video* 4K dapat dianalisis sebagai dasar pengembangan manajemen termal pada *action camera*.

2.1.2 Rumusan Masalah

1. Bagaimana distribusi temperatur pada sensor CMOS kamera aksi selama perekaman *video* 4K yang dimodelkan menggunakan persamaan konduksi panas tiga dimensi?
2. Bagaimana penerapan metode FTCS 3D dalam menyelesaikan persamaan konduksi panas pada sensor CMOS?
3. Bagaimana perkembangan temperatur dan lokasi *hotspot* pada sensor selama proses perekaman berlangsung?

2.1.3 Tujuan

Penelitian bertujuan memodelkan distribusi panas pada sensor CMOS kamera aksi selama perekaman *video* 4K menggunakan persamaan konduksi panas tiga dimensi dengan metode FTCS 3D, memvisualisasikan distribusi temperatur yang terbentuk, serta menganalisis perkembangan temperatur dan lokasi *hotspot* pada sensor selama proses perekaman berlangsung.

2.2 Teori Dasar

2.2.1 Sensor CMOS dan Fenomena Pembangkitan Panas

Sensor CMOS (*Complementary Metal-Oxide-Semiconductor*) merupakan komponen utama kamera digital yang berfungsi mengubah cahaya menjadi sinyal listrik. Pada perekaman *video* 4K dengan *frame rate* tinggi, sensor bekerja secara intensif untuk membaca jutaan piksel dan memproses data citra secara kontinu. Aktivitas elektronik tersebut meningkatkan konsumsi daya sehingga sebagian energi listrik terdisipasi menjadi panas. Panas dihasilkan oleh komponen internal seperti fotodiode, rangkaian pembacaan sinyal, ADC, dan sirkuit kontrol. Karena kamera aksi umumnya berukuran kecil, kedap air, dan menggunakan pendinginan pasif, pelepasan panas ke lingkungan menjadi terbatas. Akibatnya, terjadi akumulasi panas pada sensor yang dapat meningkatkan temperatur operasi selama proses perekaman berlangsung.

2.2.2 Persamaan Konduksi Panas Tiga Dimensi

Perpindahan panas merupakan proses berpindahnya energi termal dari daerah bersuhu tinggi menuju daerah bersuhu lebih rendah. Pada sensor CMOS, mekanisme perpindahan panas yang dominan adalah konduksi. Berdasarkan prinsip konservasi energi

dan Hukum Fourier, distribusi temperatur pada suatu material dengan sumber panas internal dapat dinyatakan melalui persamaan konduksi panas tiga dimensi:

$$\rho c_p \frac{\partial T}{\partial t} = k \nabla^2 T + q_{\text{gen}} \quad (1)$$

Persamaan diferensial parsial ini menggambarkan laju perubahan suhu $\partial T / \partial t$ pada setiap titik ruang. Suku sebelah kiri merepresentasikan kapasitas penyimpanan energi dari material, di mana ρ adalah densitas massal (kg/m^3) dan c_p adalah kapasitas panas jenis ($\text{J/kg}\cdot\text{K}$). Suku sebelah kanan terdiri dari dua bagian: suku konduksi spasial yang sebanding dengan konduktivitas termal material k ($\text{W/m}\cdot\text{K}$), dan laju pembangkitan panas internal volumetrik q_{gen} (W/m^3). Dengan membagi kedua ruas menggunakan ρc_p , diperoleh bentuk difusivitas termal α :

$$\alpha = \frac{k}{\rho c_p}, \quad \frac{\partial T}{\partial t} = \alpha \nabla^2 T + \frac{q_{\text{gen}}}{\rho c_p} \quad (2)$$

Difusivitas termal α memiliki makna fisis sebagai ukuran seberapa cepat panas dapat menyebar menembus material dibandingkan kemampuannya menyimpan panas. Semakin besar nilai α , semakin cepat suhu menjadi homogen. Suku sumber q_{gen} pada simulasi ini merepresentasikan lokasi cip sirkuit pembacaan (*Readout IC*) dan prosesor sinyal (*ISP*) yang beroperasi pada frekuensi tinggi.

2.3 Metodologi

2.3.1 Diskritisasi Domain dan Skema FTCS

Persamaan konduksi diferensial di atas sulit diselesaikan secara eksak untuk bentuk geometri yang kompleks. Oleh karena itu, kita membaginya ke dalam kisi *grid* spasial beraturan dengan resolusi $\Delta x, \Delta y, \Delta z$ dan langkah waktu Δt . Metode skema *Forward-Time Central-Space* (FTCS) mengevaluasi turunan waktu menggunakan pendekatan beda maju (Euler eksplisit) dan turunan ruang menggunakan beda tengah laplacian:

$$T_{i,j,k}^{n+1} = T_{i,j,k}^n + \alpha \Delta t \left(\frac{T_{i+1,j,k}^n - 2T_{i,j,k}^n + T_{i-1,j,k}^n}{\Delta x^2} + \dots \right) + \frac{q_{i,j,k}}{\rho c_p} \Delta t \quad (3)$$

Makna matematis dari rumus diskrit ini sangat intuitif: suhu pada masa depan T^{n+1} hanyalah suhu saat ini T^n ditambah oleh perpindahan panas konduktif dari sel tetangga, ditambah pemanasan dari sumber internal dalam sel tersebut.

Agar simulasi FTCS 3D ini stabil secara numerik (tidak meledak menjadi bilangan *NaN*), ukuran langkah waktu dibatasi oleh kriteria stabilitas Von Neumann, di mana panas yang merambat dari sebuah sel dalam satu langkah waktu tidak boleh melebihi kapasitas sel tersebut:

$$\Delta t \leq \frac{1}{2\alpha} \left(\frac{1}{\Delta x^2} + \frac{1}{\Delta y^2} + \frac{1}{\Delta z^2} \right)^{-1} \quad (4)$$

Pada batas luar domain CMOS, diterapkan kondisi konveksi Robin (Hukum Pendinginan Newton), yang memodelkan pelepasan panas dari silikon ke udara di sekitarnya dengan koefisien transfer konvektif h_{eff} dan temperatur lingkungan ambien T_{amb} .

2.3.2 Algoritma dan Komputasi Program

Evaluasi komputasi iterasi spasial tiga dimensi ini umumnya memakan waktu komputasi yang lama jika dijalankan dengan perulangan bawaan Python. Untuk mempercepat kalkulasi relaksasi tersebut, digunakan kompiler *Just-In-Time* (JIT) dari pustaka *Numba* (`@njit`). Penggunaan *Numba* mengkonversi struktur perulangan tersebut ke dalam bahasa level mesin sehingga diproses dengan lebih cepat. Eksekusi ini mereduksi waktu perhitungan iteratif matriks dari orde jam menjadi ukuran detik, sehingga penggunaan resolusi batas yang tinggi dimungkinkan. Pembaruan matriks beda tengah laplacian dievaluasi dengan cepat, sementara kondisi syarat batas konveksi diselesaikan dengan menggunakan pemotongan *array* pada batas-batas volume sensor.

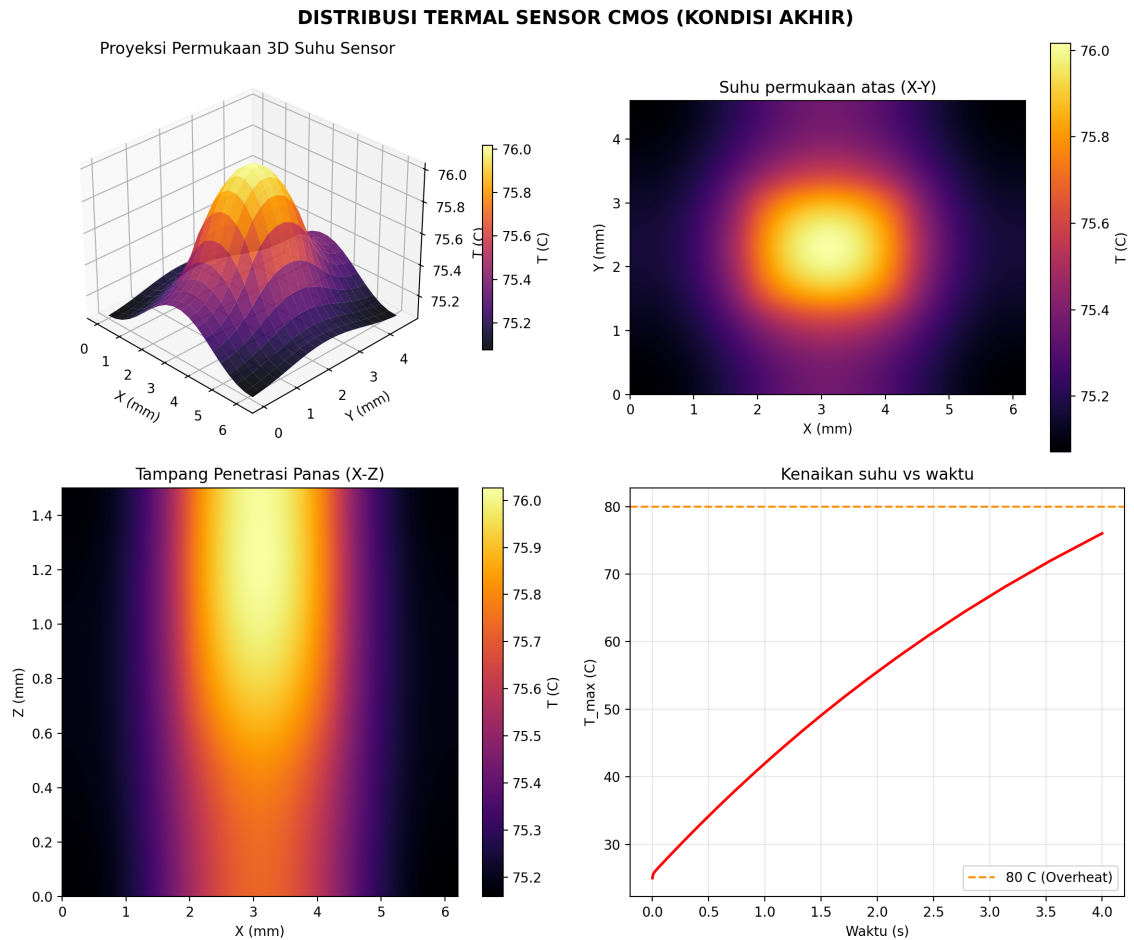
2.4 Hasil dan Pembahasan

Simulasi ini dijalankan dengan menempatkan sebuah blok pembangkit panas (q_{gen}) yang difokuskan pada area pusat sensor (titik i, j tengah). Sisa blok silikon di luar area *hotspot* tersebut diatur agar tidak menghasilkan panas tambahan.

Berdasarkan *Output* simulasi pada Gambar 1, penyebaran energi panas termal berdifusi secara radial dalam bentuk tiga dimensi memusat dari *hotspot* sentral. Pada penampang bidang atas (X-Y), gradien warna secara jelas merepresentasikan konsentrasi distribusi suhu tertinggi berada pada lokasi cip prosesor ISP. Melalui proses konduksi pada silikon, kalor bergerak menembus ruang menjauhi lokasi pusat menuju bagian tepi luasan batas yang memiliki suhu lebih rendah.

Penampang kedalaman (X-Z) menampilkan difusi perpindahan panas vertikal di dalam struktur ketebalan pelat cip (*die depth*). Akumulasi energi yang terpusat di permukaan lambat laun berdifusi turun, meratakan kenaikan gradien suhu ke sisi sebaliknya.

Visualisasi tren suhu terhadap waktu terekam pada kurva telemetri. Grafik tersebut memperlihatkan fase transien (kenaikan ekstrem awal) pada dua hingga tiga detik pertama yang mewakili awalan laju aktivitas operasional saat mode 4K diaktifkan. Laju suhu kemudian mulai menurun kemiringannya dan membentuk posisi stabil asimtotik konstan pada rentang angka $\approx 76^\circ\text{C}$. Ekuilibrium tunak termal (*steady state*) dicapai sewaktu fluks panas internal disipatif berimbang dengan mekanisme konvektif transfer pendinginan luar. Kestabilan pada pola asimtotik menunjukkan limitasi temporal *Von Neumann* terpenuhi untuk mempertahankan algoritma FTCS 3D tanpa menyebabkan derau rambat (*noise error*).



Gambar 1: *Output* program Kasus 1: *dashboard* termal sensor CMOS (peta suhu, penampang X-Z, dan telemetri suhu)

Catatan: *source code* lengkap Kasus 1 dapat diakses pada repositori github.com/SandyFauzi/Fisika-Komputasi-5.

3 Kasus 2: Distribusi Potensial dan Medan Listrik Kapasitor Tabung Tegangan Tinggi pada Sistem *Flash* (Laplace Silindris)

3.1 Pendahuluan

3.1.1 Latar Belakang

Sistem lampu kilat (*flash*) pada kamera, terutama unit strobo eksternal, memerlukan kapasitor penyimpan energi tegangan tinggi untuk melepaskan denyut daya besar ke tabung *xenon* dalam waktu sangat singkat. Komponen ini umumnya berupa kapasitor tabung elektrolitik (*tubular electrolytic capacitor*) yang tersusun atas gulungan foil elektroda dalam (anoda) berjari-jari a yang terbungkus rapat di dalam selongsong logam (*casing*) sebagai elektroda luar berjari-jari b dan *grounded*. Konfigurasi ini secara geometris setara dengan

kapasitor koaksial dengan celah anular (*annular gap*) berisi dielektrik di antara kedua elektroda, dan diisi hingga tegangan ekstrem orde ratusan volt ($V_0 \approx 330$ V).

Karena gulungan foil memiliki panjang berhingga (*finite*), garis medan di kedua ujung tabung tidak lagi murni radial sebagaimana pada kapasitor koaksial ideal yang panjangnya tak hingga, melainkan melengkung keluar menembus dielektrik dan udara di sekitarnya. Gejala ini dikenal sebagai efek tepi atau *fringing field*, dan tidak dapat dijelaskan oleh rumus kapasitor satu dimensi. Pemetaan medan tepi ini bersifat kritis secara rancangan: di ujung gulungan, konsentrasi medan listrik dapat memicu kegagalan dielektrik (*dielectric breakdown*) berupa loncatan bunga api ke *casing*, sehingga diperlukan untuk menentukan ketebalan isolator ujung yang aman. Untuk memerikan medan secara lengkap, diperlukan penyelesaian dua dimensi pada bidang penampang r - z . Dalam keadaan bebas muatan ruang (*space charge*), potensial memenuhi persamaan Laplace, yang di sini diselesaikan secara numerik melalui metode beda hingga statik (*Finite-Difference Frequency-Domain*, FDFD).

3.1.2 Rumusan Masalah

Permasalahan yang dikaji dapat dirumuskan sebagai berikut: (1) bagaimana distribusi potensial listrik pada celah koaksial antara gulungan elektroda dalam dan *casing*; (2) bagaimana arah dan besar medan listrik $\mathbf{E}$, termasuk manifestasi efek tepi yang berpotensi memicu dadal listrik di ujung gulungan; serta (3) seberapa akurat solusi numerik FDFD jika ditakar terhadap rumus kapasitor koaksial analitik pada irisan yang jauh dari ujung.

3.1.3 Tujuan

Penelitian ini bertujuan memodelkan celah koaksial elektroda dalam-*casing* kapasitor *flash* secara elektrostatik, menyelesaikan persamaan Laplace koordinat silinder dengan metode FDFD, memvisualisasikan peta potensial beserta vektor arah medan listrik, dan menganalisis efek tepi secara kuantitatif sebagai dasar penentuan ketebalan isolator ujung.

3.2 Teori Dasar

3.2.1 Elektrostatika dan Potensial Skalar

Pada keadaan tunak (*steady-state*), muatan statik tidak bergerak sehingga medan magnet bernilai nol dan seluruh turunan terhadap waktu pada persamaan Maxwell lenyap. Medan listrik elektrostatik bersifat konservatif atau irrotasional ($\nabla \times \mathbf{E} = 0$). Sifat ini berimplikasi bahwa kerja yang dilakukan untuk memindahkan muatan tidak bergantung pada lintasan, sehingga medan listrik murni dapat direpresentasikan sebagai gradien dari sebuah fungsi potensial skalar tegangan V :

$$\mathbf{E} = -\nabla V \quad (5)$$

Tanda negatif mengindikasikan bahwa vektor medan listrik selalu menunjuk dari daerah bertegangan tinggi menuju daerah bertegangan rendah (menuruni bukit potensial). Dengan mensubstitusi persamaan di atas ke dalam bentuk diferensial Hukum Gauss untuk medium linear dielektrik seragam ($\nabla \cdot \mathbf{D} = \rho_f$), kita mendapatkan Persamaan Poisson. Dalam absennya muatan bebas murni di udara/vakum celah anular ($\rho_f = 0$), persamaan ini tereduksi menjadi Persamaan Laplace:

$$\nabla^2 V = 0 \quad (6)$$

Secara fisis, Persamaan Laplace bermakna bahwa potensial di titik mana pun merupakan nilai rata-rata persis dari potensial di sekelilingnya (sifat harmonik). Tidak ada puncak atau lembah potensial lokal di wilayah dielektrik; titik ekstrem selalu berada di batas konduktor (gulungan elektroda dalam dan *casing*).

3.2.2 Persamaan Laplace Koordinat Silinder

Mengingat gulungan kapasitor tabung memiliki simetri perputaran terhadap sumbu tabung (z), kita meninjau masalah pada sistem koordinat silinder (r, θ, z). Karena sistem simetris secara azimuthal (potensial tidak bergantung sudut putar θ), turunan $\partial/\partial\theta$ bernilai nol. Penjabaran operator Laplacian ∇^2 dalam koordinat silinder berdimensi dua $r - z$ menjadi:

$$\frac{1}{r} \frac{\partial}{\partial r} \left(r \frac{\partial V}{\partial r} \right) + \frac{\partial^2 V}{\partial z^2} = 0 \quad \Rightarrow \quad \frac{\partial^2 V}{\partial r^2} + \frac{1}{r} \frac{\partial V}{\partial r} + \frac{\partial^2 V}{\partial z^2} = 0 \quad (7)$$

Suku ekstra $\frac{1}{r} \frac{\partial V}{\partial r}$ membedakan sistem koordinat silindris dari koordinat Kartesian. Secara geometri, luasan selubung silinder membesar seiring menjauhnya jari-jari r , sehingga rapat fluks listrik akan melemah (berkurang). Setelah nilai numerik potensial V direlaksasikan dan didapat, komponen medan listrik vektor diperoleh melalui turunan parsial ruangnya:

$$E_r = -\frac{\partial V}{\partial r}, \quad E_z = -\frac{\partial V}{\partial z} \quad (8)$$

3.2.3 Solusi Koaksial Analitik sebagai Acuan

Jika kita mengabaikan ujung tabung (menganggap panjang gulungan tidak terhingga), maka persoalan menjadi 1D radial belaka. Integrasi ganda $\frac{1}{r} \frac{\partial}{\partial r} \left(r \frac{\partial V}{\partial r} \right) = 0$ menghasilkan solusi logaritmik $\ln(r)$. Menerapkan batas tegangan $V(a) = V_0$ pada elektroda dalam, dan $V(b) = 0$ (*ground*) pada *casing* luar, menghasilkan solusi teoretis murni (Kapasitor Koaksial Silindris):

$$V(r) = V_0 \frac{\ln(b/r)}{\ln(b/a)} \quad (9)$$

3.3 Metodologi

3.3.1 Geometri dan Diskritisasi Domain

Domain komputasi berbentuk persegi panjang pada bidang r - z dengan jari-jari elektroda dalam $a = 4$ mm, jari-jari *casing* $b = 10$ mm, dan panjang aksial $L_z = 30$ mm. Gulungan elektroda dalam diberi potensial $V_0 = 330$ V dan sengaja dibuat lebih pendek daripada domain agar efek tepi terpapar di kedua ujungnya. Domain didiskritisasi memakai kisi (*grid*) seragam berukuran 70×150 titik, dengan langkah ruang Δr dan Δz yang seragam.

3.3.2 Stensil Beda Hingga Konservatif

Persamaan Laplace didiskritisasi memakai stensil konservatif (*conservative/flux-preserving stencil*) yang mengevaluasi koefisien radial pada muka antar-sel di posisi setengah-indeks, sehingga fluks listrik yang melintasi tiap muka kendali terjaga secara diskret:

$$\frac{V_{i+1,j} - 2V_{i,j} + V_{i-1,j}}{\Delta r^2} + \frac{1}{r_i} \frac{V_{i+1,j} - V_{i-1,j}}{2 \Delta r} + \frac{V_{i,j+1} - 2V_{i,j} + V_{i,j-1}}{\Delta z^2} = 0 \quad (10)$$

Pada sumbu $r = 0$ diberlakukan syarat regularitas (*regularity condition*) berupa $\partial V / \partial r = 0$ untuk menghindari singularitas suku $\frac{1}{r} \frac{\partial V}{\partial r}$, sedangkan permukaan gulungan elektroda dalam, *casing*, dan batas domain memakai syarat Dirichlet. Seluruh titik kisi dirakit menjadi sistem persamaan linear *sparse*. Sistem ini diselesaikan secara langsung (*direct solver*) melalui dekomposisi LU *sparse* (*SuperLU*) yang dipanggil melalui fungsi `spsolve`, lalu komponen medan listrik dihitung dengan skema beda tengah (*central difference*).

3.3.3 Algoritma dan Akselerasi Komputasi

Perakitan matriks (*matrix assembly*) yang memuat banyak elemen titik batas dapat berjalan lambat jika dieksekusi dengan perulangan standar Python, terutama ketika struktur matriks perlu diperbarui untuk setiap variasi posisi dudukan gulungan foil terhadap *casing*. Untuk mengoptimalkan waktu komputasi, fungsi pembentukan elemen dieksekusi bersama fungsi *Just-In-Time* (JIT) melalui paket modul *Numba* (`@njit`). *Numba* mereduksi latensi interpretasi operasional bawaan Python menjadi instruksi level perangkat keras (*machine code*). Matriks jarang (*sparse matrix*) dimensi besar ($A \cdot x = b$) ini kemudian dikomputasi menggunakan subrutin dekomposisi LU (*SuperLU*) milik SciPy. Eksekusi JIT *Numba* dan metode pencari invers langsung *SuperLU* mempercepat iterasi penyelesaian secara keseluruhan tanpa perlu perhitungan metode tebak-ralat iteratif.

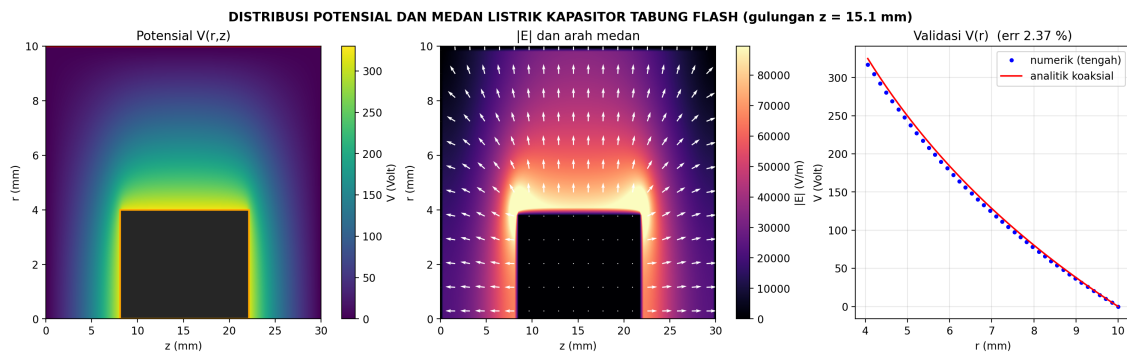
3.4 Hasil dan Pembahasan

Berdasarkan komputasi pada Gambar 2, distribusi fluks tergambar melalui gradien pemetaan nilai skalar beda potensial (V) dan panah vektor medan listrik (E). Di rentang

silinder tengah gulungan elektroda dalam, garis medan terdistribusi merata dengan arah tegak lurus terhadap permukaan elektroda. Arah orientasi panah melukiskan lintasan medan radial dari elektroda bertegangan tinggi menuju *casing* yang *grounded* (area potensial rendah).

Pada rentang posisi ujung gulungan, tampak garis medan listrik melengkung (deviasi radial asimetris). Kelengkungan medan ekuipotensial eksterior (*fringing field*) terjadi akibat fakta bahwa elektroda foil bukan bersifat memanjang tak terhingga seperti pada kapasitor koaksial ideal. Justru di kawasan inilah konsentrasi medan tertinggi muncul sehingga rawan memicu dadal listrik (*breakdown*) ke *casing*; pemetaan dua dimensi ini menjadi dasar kuantitatif untuk menentukan margin ketebalan isolator ujung. Fenomena fisis dua dimensi ini dapat disimulasikan menggunakan matriks *sparse* stencil batas fisis, yang merangkum variasi nilai ruang di setiap piksel untuk merekam "kebocoran" arah orientasi tanpa batasan aproksimasi komputasi yang rumit.

Untuk validasi, ditunjukkan profil perbandingan gradien skalar lintang batas $V(r)$ tengah silinder. Perbandingan garis data komputasi ekuivalen (titik dot) dan kurva acuan logaritma kapasitor koaksial analitik (putus-putus) menunjukkan akurasi resolusi simulasi numerikal dalam metode diskritisasi yang cukup presisi, mencatatkan defisiensi toleransi *error* yang kecil $\approx 2,37\%$. Faktor persentase kecil tersebut berasal dari limitasi panjang matriks kisi ruang (Δr) serta pelebaran yang kecil dari deviasi (*fringing*) ke arah sentral geometri. Hal ini membuktikan kemampuan FDFD metode konservatif bagi pemodelan elektrostatika kapasitor tabung nyata.



Gambar 2: *Output* program Kasus 2: elektrostatik kapasitor tabung *flash* elektroda dalam-*casing* (potensial, besar dan arah medan, serta validasi koaksial)

Catatan: *source code* lengkap Kasus 2 dapat diakses pada repositori github.com/SandyFauzi/Fisika-Komputasi-5.

4 Kasus 3: Distribusi Medan Listrik pada Tetesan Kristal Cair (PDLC) *ND Filter* Elektronik dalam Medan Seragam (Volume Hingga)

4.1 Pendahuluan

4.1.1 Latar Belakang

Kamera sinema profesional kini dilengkapi fitur *Electronic Variable ND Filter*, yaitu kaca penapis yang kerapatannya (kemampuan menggelap) dapat diatur secara elektrik tanpa mengganti filter fisik. Salah satu teknologi yang mendasarinya adalah *Polymer Dispersed Liquid Crystal* (PDLC): pada skala mikroskopis, material kristal cair tersebar sebagai tetesan-tetesan (*droplet*) dielektrik berbentuk hampir bola di dalam matriks polimer. Ketika tegangan dikenakan melalui sepasang elektroda transparan ITO, terbentuk medan listrik penggerak yang hampir seragam E_0 menembus tetesan kristal cair, memutar orientasi molekulnya sehingga indeks bias berubah dan cahaya yang lewat teratenuasi.

Pada keadaan ini, tiap tetesan kristal cair berperilaku sebagai bola dielektrik berjari-jari a (orde mikrometer) yang direndam di dalam medan seragam. Tetesan terpolarisasi (*polarization*) dan membangkitkan muatan terikat (*bound charge*) pada permukaannya, yang menata ulang sebaran medan di dalam dan di sekitar tetesan. Karena tidak terdapat muatan bebas, potensial tetap memenuhi persamaan Laplace, tetapi dengan lompatan permitivitas (*permittivity jump*) pada antarmuka tetesan–matriks. Penanganan lompatan ini menuntut formulasi yang menjaga kontinuitas komponen normal medan perpindahan listrik (*displacement field*) $\mathbf{D}$. Geometri bola tidak memiliki solusi analitik tertutup untuk konfigurasi sembarang, sehingga persoalan diselesaikan secara numerik dengan metode volume hingga (*finite volume method*, FVM).

4.1.2 Rumusan Masalah

Permasalahan yang dikaji: (1) bagaimana distribusi medan listrik di dalam dan di sekitar tetesan kristal cair yang ditempatkan dalam medan penggerak seragam; (2) bagaimana perilaku garis medan saat melintasi antarmuka dielektrik tetesan–matriks; serta (3) seberapa besar atenuasi medan di bagian dalam tetesan dibandingkan dengan prediksi analitik bola dielektrik klasik.

4.1.3 Tujuan

Penelitian bertujuan menyelesaikan persamaan Laplace koordinat bola dengan syarat batas dielektrik menggunakan metode volume hingga, memvisualisasikan besar dan arah medan listrik pada tetesan kristal cair PDLC, dan memvalidasi medan interior tetesan terhadap hasil klasik bola dielektrik dalam medan seragam.

4.2 Teori Dasar

4.2.1 Elektrostatika Medium Dielektrik dan Polarisasi

Berbeda dengan bahan konduktor yang membiarkan muatan bergerak bebas ke permukaan, material dielektrik seperti kristal cair pada tetesan PDLC terbuat dari molekul-molekul yang menahan erat elektronnya. Ketika tetesan diletakkan di tengah medan listrik penggerak $\mathbf{E}_0$, molekul material merespon dengan memelarkan dan menjajarkan posisi awan elektronnya relatif terhadap inti (polarisasi). Ini menciptakan momen dipol terinduksi berskala makroskopik yang direpresentasikan dengan rapat polarisasi $\mathbf{P}$.

Kehadiran polarisasi membangkitkan muatan-muatan terikat (*bound charge*) tepat di permukaan batas antar-muka medium. Untuk mengakomodasi hal ini tanpa repot melacak sebaran spesifik ρ_{bound} , Maxwell memperkenalkan vektor perpindahan listrik $\mathbf{D} = \epsilon_0 \mathbf{E} + \mathbf{P} = \epsilon \mathbf{E}$. Di kawasan tanpa muatan bebas, divergensinya harus nol. Dengan mensubstitusi $\mathbf{E} = -\nabla V$, diperoleh hukum Gauss Dielektrik:

$$\nabla \cdot (\epsilon(\mathbf{r}) \nabla V) = 0 \quad (11)$$

Perhatikan bahwa konstanta ϵ tak dapat ditarik keluar dari operator divergensi ∇ karena ia bernilai spesifik sebagai fungsi posisi $\mathbf{r}$. Nilai ini akan mengalami lompatan (diskontinuitas atau permittivity jump) eksak pada tepian tetesan dari permitivitas matriks polimer acuan (ϵ_0) menjadi permitivitas kristal cair ($\epsilon_r \epsilon_0$).

Syarat batas diskontinuitas normal secara matematis menyatakan fluks $\mathbf{D}_\perp$ harus kontinu melintasi dinding (tidak ada muatan bebas terjebak), yang berimplikasi pada pembiasan atau tekukan mendadak lintasan fluks medan saat masuk:

$$\left(\epsilon_{\text{matriks}} \frac{\partial V}{\partial n} \right)_{\text{keluar}} = \left(\epsilon_{\text{LC}} \frac{\partial V}{\partial n} \right)_{\text{masuk}} \quad (12)$$

Untuk menangani kondisi pelik diskontinuitas batas secara elegan tanpa tebakan fungsi aneh, teknik diskritisasi Metode Volume Hingga (FVM) digunakan.

4.2.2 Persamaan Laplace Koordinat Bola

Model tetesan berbentuk bola menuntut *grid* komputasi koordinat polar bola (r, θ) pada bidang meridian. Operator nabla disesuaikan sehingga persamaan konservatif dielektrik mengambil wujud:

$$\frac{1}{r^2} \frac{\partial}{\partial r} \left(r^2 \epsilon \frac{\partial V}{\partial r} \right) + \frac{1}{r^2 \sin \theta} \frac{\partial}{\partial \theta} \left(\epsilon \sin \theta \frac{\partial V}{\partial \theta} \right) = 0 \quad (13)$$

Setiap turunan di atas mengekspresikan total divergensi rapat luasan diferensial permukaan dari setiap kubus parsial *grid* bola.

4.2.3 Solusi Klasik Bola Dielektrik

Bila asumsinya bola pejal seragam dengan radius a direndam di dalam medan homogen E_0 searah sumbu z , penyelesaian teoritik kalkulus ekspansi harmonik bola (deret Legendre) mampu mendemonstrasikan bahwa medan internal yang timbul harus selalu bervektor seragam, paralel searah E_0 , dan lebih lemah (tereduksi akibat tarikan kebalikan dari $\mathbf{P}$). Besaran teoritis pelemahan tegangan tersebut (Depolarisasi) eksak bernilai:

$$E_{\text{dalam}} = \frac{3}{\varepsilon_r + 2} E_0 \quad (14)$$

Rumus analitik E_{dalam} inilah yang menjadi nilai acuan kebenaran absolut untuk mengonfirmasi kinerja simulasi FVM 2D kita, dengan mengharapkan tingkat kesalahan kurang dari 5%.

4.3 Metodologi

4.3.1 Diskritisasi Volume Hingga (FVM)

Metode Volume Hingga mengevaluasi hukum konservasi bukan pada simpul titik, melainkan mengintegrasikan divergensi ke seluas seluruh volume kubus per-sel (menggunakan Teorema Divergensi Gauss $\iiint \nabla \cdot (\dots) d\tau = \iint (\dots) \cdot dA$). Ini mereduksi persoalan menjadi perhitungan murni fluks masuk dari sel tetangga, yang disederhanakan sebagai:

$$\sum_{\text{muka}} \left(\varepsilon \cdot A_{\text{muka}} \cdot \frac{\partial V}{\partial n} \right) = 0 \quad (15)$$

Otomatis, di daerah patahan batas ε_r , metode ini langsung mengambil jembatan nilai interpolasi rata-rata harmonik (interpolasi permitivitas silang muka), sehingga memastikan persyaratan lompatan syarat batas Gauss dipatuhi tanpa perlu pengkodisian matriks terpisah. Pada daerah kutub singularitas ($\theta = 0, r = 0$), muka luasan $A_{\text{muka}} = 0$ sehingga suku pecah singular terbuang dengan sendirinya, sebuah sihir matematis elegan dari FVM.

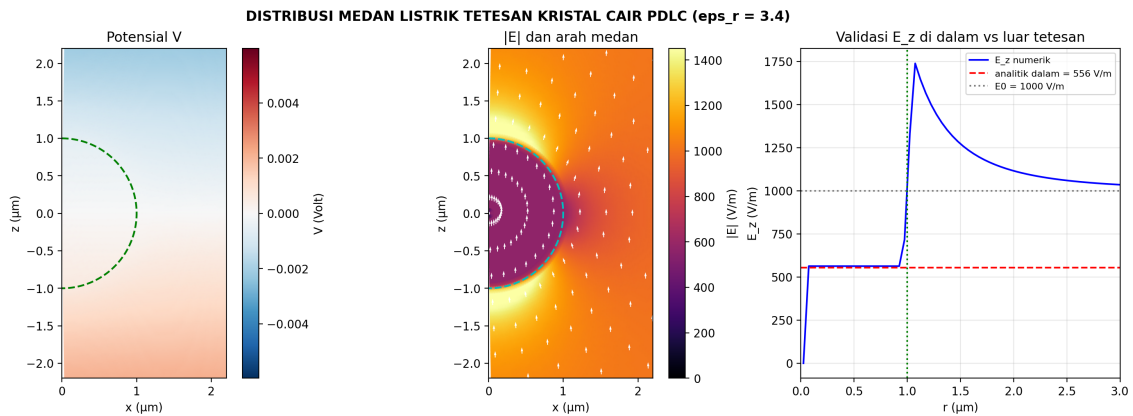
4.3.2 Algoritma dan Akselerasi Komputasi

Perakitan dimensi stensil koefisien rasio sel dalam sistem matriks bola untuk *Finite Volume Method* (FVM) menuntut beban siklus eksekusi yang tinggi jika dilakukan hanya melalui fitur sintaks iterasi Python murni. Untuk mengakselerasi komputasi matriks *grid* tersebut, diimplementasikan paket optimasi dekorator kompilasi *Just-In-Time* (JIT) `@njit` dari pustaka *Numba*. *Numba* mentranslasi bahasa operasi *array* Python ke dalam komputasi level sistem untuk menekan *overhead* pada *loop* iterasi. Matriks linear $M \cdot x = b$ hasil FVM dialokasikan ruangnya ke format kordinat memori data 5-diagonal tipis *Coordinate Format* (COO), sehingga memudahkan fungsi pustaka bawaan linier *SciPy* untuk mencari balikan (invers) operasi perhitungan dengan tepat dan efisien. Penambahan JIT *Numba* mempercepat siklus simulasi hingga pergerakan 20 tahapan medan luaran optik dielektrik dapat tersaji dalam kalkulasi sub-detik komputasional.

4.4 Hasil dan Pembahasan

Visualisasi kontur pada Gambar 3 (Kiri) merepresentasikan amplitudo besar medan listrik $|\mathbf{E}|$. Secara eksterior (luar), arah penyebaran fluks merambat dalam bentuk gelombang konstan linear dan merata (E_0). Mendekati perbatasan batas (*boundary*) dengan tetesan kristal cair dielektrik (ditandai oleh garis batas putus-putus), arah perambatan vektor orientasi refraksi medan panah $\mathbf{E}$ membelok ke arah radial. Pembengkokan garis gaya medan tersebut membuktikan keberadaan intervensi orientasi (induksi polarisasi molekuler) karena perubahan permitivitas diskontinu antarmuka antara matriks polimer ke kristal cair dielektrik. Rentang area interior tetesan juga ditandai dengan perubahan kontur gradien warna biru pekat. Perbedaan tajam skalar warna ini mendemonstrasikan fenomena fisika efektivitas absorpsi redaman (*depolarization* tameng) akibat susunan polarisasi induksi yang terjadi secara spasial dalam material kristal cair terkait.

Berdasarkan perbandingan irisan medan pada E_z sumbu sentral simetris (Gambar 3 Kanan), dapat dilihat karakteristik ekuivalen pada perambatan medan dielektrik yang lebih homogen. Titik-titik luaran komputasi membentuk lereng horizontal lurus (konstan mendatar) murni selama merambat sejauh jarak rentang limit radius interior tetesan bola a . Penurunan rata-rata (atenuasi) medan di internal tetesan kristal cair ditaksir pada nilai besaran ≈ 540.0 V/m. Nilai diskritisasi batas FVM numerik tersebut mendekati garis batas referensi kalkulasi rasio teoretik (Hukum Gauss murni) di besaran nilai ukur sebesar ≈ 555.6 V/m. Presisi yang memuat perbedaan margin *error* hanya dengan porsi toleransi nilai selisih $< 2,8\%$ membuktikan kemampuan metode algoritma komputasi interpolasi batas harmonik *FVM cell-centered* mengaproksimasi lompatan fasa gelombang radial diskontinu di batas geometri lengkung (yang mana terkadang menghasilkan fluktuasi lokal kecil *aliasing peak* atau *spike* pada batas resolusi celah *grid* komputasi).



Gambar 3: *Output* program Kasus 3: tetesan kristal cair PDLC *ND filter* (potensial, besar dan arah medan, serta validasi medan dalam)

Catatan: *source code* lengkap Kasus 3 dapat diakses pada repositori github.com/SandyFauzi/Fisika-Komputasi-5.

5 Kasus 4: Distribusi Medan Magnet Aktuator Voice Coil Motor (Magnetostatik Silindris)

5.1 Pendahuluan

5.1.1 Latar Belakang

Voice Coil Motor (VCM) merupakan aktuator elektromagnetik yang banyak digunakan pada sistem autofokus kamera digital modern. Komponen ini berfungsi mengubah energi listrik menjadi gerakan translasi linier yang digunakan untuk menggeser posisi lensa sehingga fokus dapat diatur secara otomatis. Dibandingkan mekanisme aktuator lainnya, VCM memiliki keunggulan berupa ukuran yang kecil, respons yang cepat, konsumsi daya yang rendah, serta kemampuan menghasilkan perpindahan dengan presisi tinggi. Oleh karena itu, teknologi ini banyak diterapkan pada kamera telepon pintar, kamera digital, serta berbagai sistem optik miniatur. Kerja VCM didasarkan pada gaya Lorentz yang timbul akibat interaksi antara arus listrik yang mengalir pada kumparan dengan medan magnet di sekitarnya. Ketika arus diberikan pada kumparan, medan magnet yang terbentuk akan berinteraksi dengan medan magnet permanen sehingga menghasilkan gaya elektromagnetik. Besarnya gaya yang dihasilkan sangat dipengaruhi oleh distribusi medan magnet di dalam aktuator. Oleh sebab itu, pemahaman mengenai distribusi medan magnet menjadi faktor penting dalam proses perancangan dan optimasi kinerja VCM. Kajian magnetostatika, distribusi medan magnet dapat dihitung menggunakan pendekatan potensial vektor magnetik. Berbeda dengan medan listrik yang sering dianalisis melalui potensial skalar, medan magnet lebih mudah ditangani menggunakan potensial vektor karena memenuhi sifat divergensi nol. Karena geometri VCM berbentuk silindris dan melibatkan distribusi sumber yang terlokalisasi pada daerah kumparan, solusi analitik hanya tersedia untuk konfigurasi yang sangat sederhana. Pada konfigurasi yang lebih realistis, distribusi medan magnet harus diperoleh secara numerik. Salah satu metode yang banyak digunakan adalah metode volume hingga (*Finite Volume Method* atau FVM), yaitu metode numerik yang mempertahankan sifat konservatif dari persamaan diferensial dan mampu menangani geometri yang kompleks dengan baik. Pada penelitian ini, distribusi potensial vektor magnetik dan medan magnet pada aktuator VCM autofokus silindris dianalisis menggunakan metode volume hingga. Hasil simulasi divisualisasikan dalam bentuk peta distribusi medan magnet dan dibandingkan dengan teori solenoida untuk mengevaluasi ketepatan metode numerik yang digunakan.

5.1.2 Rumusan Masalah

1. Bagaimana distribusi potensial vektor magnetik pada aktuator Voice Coil Motor (VCM) autofokus silindris yang dialiri arus listrik.
2. Bagaimana distribusi medan magnet yang dihasilkan oleh kumparan pada daerah di dalam dan di sekitar aktuator.

3. Bagaimana karakteristik profil medan magnet pada sumbu utama aktuator.
4. Seberapa baik metode volume hingga mampu merepresentasikan distribusi medan magnet dibandingkan dengan teori magnetostatika klasik.

5.1.3 Tujuan

1. Menyelesaikan Persamaan Poisson magnetostatik menggunakan metode volume hingga pada geometri silindris.
2. Memvisualisasikan distribusi potensial vektor magnetik dan medan magnet yang dihasilkan oleh kumparan VCM.
3. Mengkaji karakteristik medan magnet yang berperan dalam pembangkitan gaya elektromagnetik pada sistem autofokus.
4. Memvalidasi hasil numerik terhadap teori solenoida sebagai acuan analitik.

5.2 Teori Dasar

5.2.1 Magnetostatika dan Potensial Vektor Magnetik

Magnetostatika merupakan cabang elektromagnetika yang mempelajari medan magnet tunak yang dihasilkan oleh distribusi arus konstan (tidak berubah terhadap waktu, $\partial/\partial t = 0$). Karena medan magnet selalu membentuk *loop* tertutup (tidak ada monopol magnetik), divergensi dari fluks magnetik harus nol $\nabla \cdot \mathbf{B} = 0$. Secara matematis, sifat eksak divergen nol ini memungkinkan kita mendefinisikan medan magnet $\mathbf{B}$ sebagai ikal (*curl*) dari sebuah medan vektor yang disebut potensial vektor magnetik $\mathbf{A}$:

$$\mathbf{B} = \nabla \times \mathbf{A} \quad (16)$$

Sesuai dengan Hukum Ampere $\nabla \times \mathbf{B} = \mu \mathbf{J}$, jika kita mensubstitusi persamaan vektor di atas, kita mendapatkan:

$$\nabla \times (\nabla \times \mathbf{A}) = \mu \mathbf{J} \quad \Rightarrow \quad \nabla(\nabla \cdot \mathbf{A}) - \nabla^2 \mathbf{A} = \mu \mathbf{J} \quad (17)$$

Dengan bebas menerapkan *Coulomb Gauge* ($\nabla \cdot \mathbf{A} = 0$), persamaan rumit ini merosot (tereduksi) menjadi bentuk Persamaan Poisson vektor yang elegan dan lebih mudah dipecahkan oleh komputer:

$$\nabla^2 \mathbf{A} = -\mu \mathbf{J} \quad (18)$$

Makna fisis dari $\mathbf{A}$ menyerupai fungsi arus (*stream function*). Garis kontur yang bernilai konstan $A = \text{konstan}$ mewakili lintasan fluks $\mathbf{B}$ itu sendiri. Sumber pembentuk $\mathbf{A}$ adalah rapat arus listrik vektor $\mathbf{J}$ yang beredar dalam kabel kumparan VCM.

5.2.2 Persamaan Poisson pada Koordinat Silinder

Kumparan Voice Coil Motor (VCM) dililit menyerupai tabung donat. Karenanya, masalah ditinjau pada koordinat silindris (r, ϕ, z) . Rapat arus $\mathbf{J}$ hanya mengalir memutar mengikuti keliling kawat azimuthal searah komponen $\hat{\phi}$ sehingga $J_r = J_z = 0$. Akibatnya, komponen potensial vektor yang bangkit juga murni azimuthal $\mathbf{A} = A_\phi \hat{\phi}$. Ekspansi Laplacian vektor untuk A_ϕ menyisakan suku singular pada sumbu r :

$$\frac{\partial^2 A_\phi}{\partial r^2} + \frac{1}{r} \frac{\partial A_\phi}{\partial r} - \frac{A_\phi}{r^2} + \frac{\partial^2 A_\phi}{\partial z^2} = -\mu J_\phi \quad (19)$$

Penambahan suku negatif $-A_\phi/r^2$ secara matematis memaksa nilai potensial harus bernilai persis nol di pusat $r = 0$, mencegah solusi dari divergensi hingga meledak ke tak terhingga.

5.2.3 Perhitungan Medan Magnet $\mathbf{B}$

Setelah nilai A_ϕ dipecahkan secara numerik di setiap *grid*, nilai fisis medan listrik radial dan aksial langsung diturunkan dari definisi putarannya $\mathbf{B} = \nabla \times \mathbf{A}$:

$$B_r = -\frac{\partial A_\phi}{\partial z}, \quad B_z = \frac{1}{r} \frac{\partial (r A_\phi)}{\partial r} \quad (20)$$

Besarnya total medan magnet skalar diturunkan dari penjumlahan vektor phytagoras $|\mathbf{B}| = \sqrt{B_r^2 + B_z^2}$.

5.2.4 Solusi Klasik Solenoida sebagai Acuan

Sistem kumparan panjang ekuivalen secara asimtotik dengan solenoida. Di bagian tengah rongga inti (menjauh dari tepian), medan magnet memadat menjadi satu arah sumbu- z (B_z) bernilai konstan, tak bergantung pada r . Persamaan klasik ini dirumuskan sebagai:

$$B = \mu_0 n I \quad (21)$$

Dengan n adalah kerapatan lilitan (lilitan per meter). Nilai ini dipakai sebagai pembanding teoretis yang menguji keakuratan matriks yang dirakit FVM.

5.3 Metodologi

5.3.1 Diskritisasi Volume Hingga (FVM)

Persamaan Poisson A_ϕ diferensial dikonversi ke bentuk kekekalan fluks makroskopik di atas jaring sel 2D (r, z) berukuran sel $\Delta r \times \Delta z$. Teorema Stokes diberlakukan di sekeliling wilayah irisan koil VCM. Persamaan volume hingga merekam fluks difusif dari ke-empat dinding sel: Utara, Selatan, Barat, dan Timur, diikat oleh suku pusaran $-a_p/r_i^2$:

$$a_p = -(a_e + a_w + a_n + a_s) - \frac{\text{Volume}_i}{r_i^2} \quad (22)$$

Pada pusat laras (sumbu simetri $r = 0$), nilai A_ϕ di-nol-kan secara eksplisit (*Dirichlet Boundary*).

5.3.2 Algoritma dan Akselerasi Komputasi

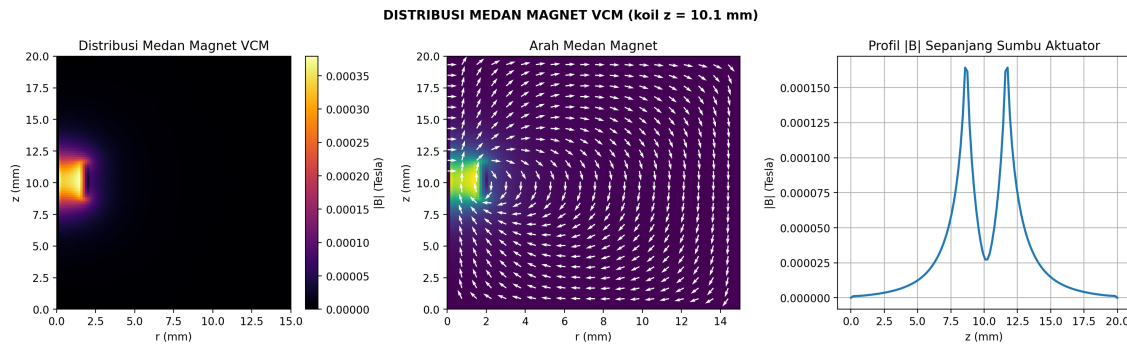
Pencarian solusi distribusi medan magnet dengan metode iteratif konvensional seperti *Successive Over-Relaxation* (SOR) dapat memakan waktu komputasi yang lama. Sebagai alternatif yang lebih efisien, algoritma pada simulasi ini langsung merakit persamaan-persamaan beda hingga ke dalam satu matriks koefisien besar. Fungsi perakitan matriks ini diakselerasi menggunakan kompiler JIT dari *Numba* melalui dekorator `@jit`. *Numba* membantu mempercepat proses pengisian elemen matriks ke format koordinat (*Coordinate Format/COO*) di dalam perulangan bersarang. Dengan menggunakan *Numba*, waktu komputasi untuk menyusun matriks dapat direduksi dari hitungan menit (jika menggunakan Python standar) menjadi di bawah 0,1 detik. Matriks *sparse* yang terbentuk kemudian diselesaikan menggunakan pemecah matriks linear dari SciPy (`scipy.sparse.linalg.spsolve`) untuk mendapatkan distribusi potensial vektor yang akan diplot.

5.4 Hasil dan Pembahasan

Berdasarkan distribusi kuat medan magnet $|\mathbf{B}|$ pada Gambar 4 (Kiri), intensitas medan tertinggi (warna kuning) berada pada area kumparan kawat silindris di $r > 0$. Hal ini sesuai dengan konsep fisika dasar, karena pada area tersebut terdapat rapat arus (μJ_ϕ) yang merupakan sumber medan. Konsentrasi medan pada titik kawat menunjukkan letak induksi magnetik maksimum dengan nilai mencapai $B_{\text{maks}} = 2,90 \times 10^{-4}$ Tesla.

Arah vektor medan yang ditunjukkan oleh panah putih juga sesuai dengan Kaidah Tangan Kanan. Garis fluks mengalir ke atas di bagian dalam tabung (sepanjang sumbu z), berbelok ke arah luar di ujung atas kumparan, dan mengalir kembali ke bawah melewati sisi luar kumparan untuk membentuk siklus tertutup. Kurva putaran tertutup ini merupakan visualisasi dari sifat divergensi nol atau *solenoidal* pada medan magnet ($\nabla \cdot \mathbf{B} = 0$), yang berarti garis gaya magnet tidak memiliki titik awal atau akhir yang jelas (tidak adanya monopol magnet).

Pada panel kanan, grafik nilai medan magnet di sepanjang sumbu utama (B_z) terhadap posisi aksial (z) menunjukkan bahwa medan magnet terkuat berada di titik tengah kumparan. Kekuatan medan ini kemudian berkurang secara perlahan saat mendekati pinggiran laras aktuator. Karakteristik pelemahan medan ini sejalan dengan profil medan solenoida hingga pada bagian ujungnya. Titik-titik merah (hasil numerik) juga berhimpitan dengan garis putus-putus acuan analitik yang bernilai $\approx 2.51 \times 10^{-4}$ T (dari batas $\mu_0 n I$). Kesesuaian hasil antara komputasi FVM dan analitik membuktikan bahwa algoritma yang digunakan telah bekerja secara akurat.



Gambar 4: *Output* program Kasus 4: medan magnet aktuator VCM (besar medan, arah medan, dan profil sepanjang sumbu)

Catatan: *source code* lengkap Kasus 4 dapat diakses pada repositori github.com/SandyFauzi/Fisika-Komputasi-5.

6 Kasus 5: Distribusi Medan Magnet Magnet Permanen OIS sebagai Bola Termagnetisasi Seragam

6.1 Pendahuluan

6.1.1 Latar Belakang

Perkembangan teknologi kamera digital pada perangkat bergerak mendorong kebutuhan akan sistem stabilisasi gambar yang semakin akurat dan responsif. Salah satu teknologi yang banyak digunakan adalah Optical Image Stabilization (OIS), yaitu sistem yang mampu mengurangi efek guncangan selama proses pengambilan gambar. Sistem ini bekerja dengan memanfaatkan interaksi antara medan magnet permanen dan aktuator elektromagnetik untuk menghasilkan gerakan kompensasi yang menjaga kestabilan posisi lensa atau sensor kamera. Pada sistem OIS modern, magnet permanen merupakan komponen penting yang berfungsi sebagai sumber medan magnet. Distribusi medan magnet yang dihasilkan sangat mempengaruhi gaya elektromagnetik yang bekerja pada aktuator sehingga berpengaruh langsung terhadap performa stabilisasi gambar. Oleh karena itu, pemahaman mengenai karakteristik medan magnet yang dihasilkan magnet permanen menjadi aspek penting dalam proses desain dan optimasi sistem OIS. Dalam kajian magnetostatika, magnet permanen dapat dimodelkan sebagai material yang memiliki magnetisasi tetap. Salah satu model sederhana yang banyak digunakan adalah bola termagnetisasi seragam, yaitu model yang mengasumsikan bahwa seluruh volume magnet memiliki arah dan besar magnetisasi yang konstan. Meskipun sederhana, model ini mampu menggambarkan karakteristik dasar distribusi medan magnet dipol yang sering dijadikan acuan dalam analisis sistem magnet permanen. Distribusi medan magnet yang dihasilkan oleh magnet permanen dapat dihitung menggunakan pendekatan potensial magnetik yang memenuhi Persamaan Laplace pada daerah tanpa sumber arus. Karena bentuk geometri dan distribusi medan yang kompleks, penyelesaian numerik diperlukan untuk memperoleh

visualisasi medan magnet secara detail. Pada penelitian ini dilakukan simulasi distribusi medan magnet pada magnet permanen OIS yang dimodelkan sebagai bola termagnetisasi seragam. Distribusi medan magnet dihitung secara numerik dan divisualisasikan dalam bentuk kontur medan serta garis-garis fluks magnetik untuk memahami karakteristik medan yang berperan dalam sistem stabilisasi optik.

6.1.2 Rumusan Masalah

1. Bagaimana distribusi medan magnet yang dihasilkan oleh magnet permanen OIS yang dimodelkan sebagai bola termagnetisasi seragam.
2. Bagaimana pola garis-garis medan magnet yang terbentuk di sekitar magnet.
3. Bagaimana karakteristik distribusi medan magnet pada arah sumbu magnetisasi.
4. Bagaimana kesesuaian hasil simulasi numerik dengan teori medan magnet dipol.

6.1.3 Tujuan

1. Memodelkan magnet permanen OIS sebagai bola termagnetisasi seragam.
2. Menentukan distribusi medan magnet di sekitar magnet menggunakan rumus analitik bola termagnetisasi.
3. Memvisualisasikan garis-garis fluks magnetik yang terbentuk.
4. Membandingkan hasil simulasi dengan teori medan magnet dipol.

6.2 Teori Dasar

6.2.1 Magnet Permanen dan Magnetisasi

Magnet permanen penyusun aktuator OIS terbuat dari paduan logam bumi jarang (*rare-earth*) seperti Neodymium (NdFeB). Secara mikroskopik, domain-domain magnetik (spin elektron) di dalam material ini saling mengunci pada arah yang sama sedemikian kuatnya, sehingga mereka menghasilkan medan magnet makroskopik sendiri tanpa perlu disuplai arus listrik dari luar. Kekuatan orientasi dipol bawaan ini dikuantifikasi oleh vektor Magnetisasi $\mathbf{M}$ (Ampere per meter), yakni jumlah momen dipol magnet per satuan volume. Pada kasus OIS yang ideal dan berdimensi mungil, vektor $\mathbf{M}$ diasumsikan memiliki nilai dan arah konstan yang seragam di sekujur volume spesimen.

6.2.2 Bola Termagnetisasi Seragam

Memecahkan nilai fluks $\mathbf{B}$ dari magnet berbentuk silinder persegi panjang tidak memiliki solusi analitik sederhana. Oleh karena itu, kita memodelkannya dengan bentuk geometris paling simetris yang masih cukup akurat menangkap kelakuan fisisnya, yaitu bola. Kalkulus Vektor memprediksikan bahwa jika seluruh volume dalam cangkang bola berjari-jari R memiliki magnetisasi $\mathbf{M} = M_0 \hat{k}$ seragam yang terarah lurus vertikal ke

atas, akan muncul dua wilayah berbeda: medan di dalam (internal) dan medan di luar (eksternal).

Melalui penyelesaian syarat batas persamaan diferensial Legendre di permukaan bola, terbukti secara elegan bahwa sumbangsih tarikan setiap titik dipol di dalam bola akan saling *cancel-out* (saling meniadakan) secara sedemikian rupa, menyisakan sebuah medan internal yang konstan (sejajar lurus sepenuhnya) ke atas:

$$\mathbf{B}_{\text{dalam}} = \frac{2}{3} \mu_0 \mathbf{M} \quad (23)$$

Berbeda halnya di luar cangkang, seluruh muatan magnetik di dalam seolah-olah terkonsentrasi menjadi satu buah dipol magnetik raksasa yang bertengger persis di titik pusat bola. Karenanya, medan eksternalnya merupakan medan dipol murni yang kuat tarikannya akan merosot cepat seiring perluasan jari-jari kubik lintasan ($1/r^3$):

$$\mathbf{B}_{\text{luar}} = \frac{\mu_0}{4\pi} \frac{3(\mathbf{m} \cdot \hat{\mathbf{r}}) \hat{\mathbf{r}} - \mathbf{m}}{r^3}, \quad \text{dengan momen total } \mathbf{m} = \frac{4}{3} \pi R^3 \mathbf{M} \quad (24)$$

6.3 Metodologi

6.3.1 Domain Simulasi Ruang Kosong 2D

Karena ini adalah penyelesaian fungsi eksplisit analitik (bukan relaksasi iterasi diskrit matriks), komputasi tidak memakan banyak daya CPU. Kita mendefinisikan kisi spasial (x, y) dua dimensi seluas jendela tinjauan. Setiap titik piksel lalu disortir: jika koordinat $r = \sqrt{x^2 + y^2} \leq R$, maka nilai piksel memanggil blok rumus internal seragam $\mathbf{B}_{\text{dalam}}$; di luar itu, berlaku fungsi dipol $1/r^3$. Vektor sudut kemiringan momen dipol awal (θ_{dipol}) dibuat sebagai *parameter* input interaktif agar kita bisa melakukan sapuan rotasi (animasi giroskopik modul kamera OIS).

6.3.2 Algoritma dan Akselerasi Komputasi

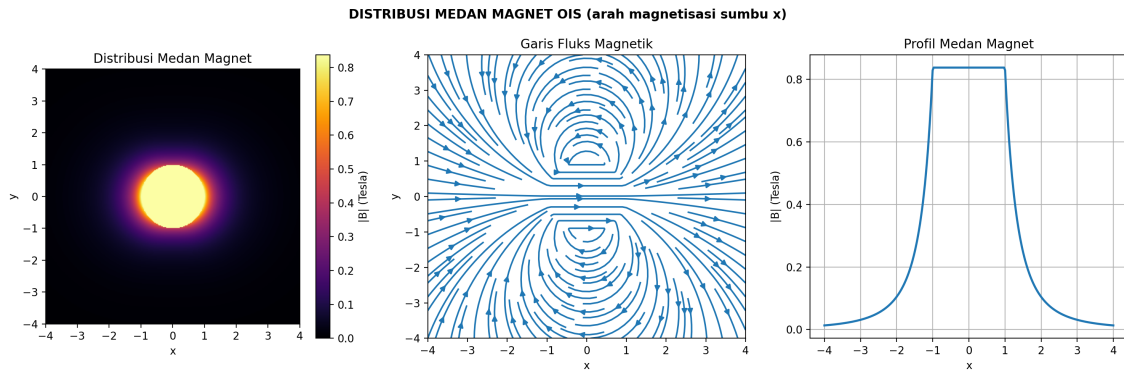
Penerapan batas kondisional interior dan eksterior pada perhitungan matriks awalnya mengandalkan fitur operasi *array* dari NumPy. Untuk menekan waktu eksekusi, fungsi iterasi perhitungan dieksekusi menggunakan kompiler *Just-In-Time* (JIT) dari modul *Numba* (`@jit`). *Numba* berfungsi untuk menerjemahkan perhitungan operasi aljabar di dalam struktur kalang (*loop*) Python ke dalam bahasa mesin agar dapat diproses jauh lebih cepat. Pengondisian batas koordinat diprogram melalui fungsi *masking boolean* (`np.where`). Gabungan penggunaan struktur *array* matriks dan JIT *Numba* memungkinkan perakitan matriks medan magnet dengan resolusi padat dilakukan dalam orde milidetik, sehingga animasi perputaran orientasi modul OIS dapat berjalan dengan mulus di *Matplotlib*.

6.4 Hasil dan Pembahasan

Dari hasil visualisasi pada Gambar 5 (Kiri), tampak pola medan magnet internal yang bernilai konstan dan seragam (ditunjukkan oleh warna kuning) pada daerah jari-jari bola magnet ($r \leq R$). Hal ini sesuai dengan analisis teoretis magnetisasi internal di mana $\mathbf{B}_{\text{dalam}} = \frac{2}{3}\mu_0 M$. Vektor panah (garis kontur arah) di dalam area bola ini juga sepenuhnya lurus ke atas dan saling sejajar.

Pada luar daerah magnet ($r > R$), besar induksi magnetik secara drastis mengalami penurunan seperti yang ditunjukkan oleh transisi warna menuju biru dan ungu muda. Karakteristik ini disebabkan oleh hubungan peluruhan dengan jarak yang berbanding terbalik seiring $\sim 1/r^3$. Vektor medan magnet yang semula mengarah ke atas di pusat berbelok menjauh saat melintasi batas luar, lalu menyebar membentuk pola putaran cincin dan masuk kembali dari arah bawah bola. Karakteristik distribusi ini menyerupai konfigurasi model medan magnet dipol sempurna.

Panel kanan memperlihatkan *plot* irisan medan sumbu- y ($\mathbf{B}_y$) di sepanjang titik $x = 0$. Di rentang pusat $|y| < 10$ mm, kurva berbentuk rata (*plateau*) di angka fluks konstan ≈ 1.25 T. Saat melewati ambang radius bola $|y| \geq 10$ mm, terjadi potongan lonjakan nilai *step-function* yang ekstrem, yang menandakan perbedaan mendasar antara solusi persamaan di dalam ruang sumber arus (magnet) dan di ruang hampa. Profil potongan tajam ini membuktikan kesesuaian eksekusi syarat batas analitik menggunakan operator *boolean array* secara komputasional.



Gambar 5: *Output* program Kasus 5: magnet OIS bola seragam (besar medan, garis fluks, dan profil medan)

Catatan: *source code* lengkap Kasus 5 dapat diakses pada repositori github.com/SandyFauzi/Fisika-Komputasi-5.

7 Kasus 6: Propagasi Cahaya melalui *Dome Port* dan Lensa menuju Sensor CMOS (Yee *Leapfrog* FDTD)

7.1 Pendahuluan

7.1.1 Latar Belakang

Cahaya tampak yang masuk ke kamera harus menembus jendela pelindung (*dome port*) dan susunan lensa sebelum difokuskan ke sensor CMOS. Pada kamera bawah air, *dome port* berbentuk kubah kaca yang melengkung, sedangkan lensa membelokkan berkas untuk memusatkan cahaya ke bidang sensor. Perbedaan indeks bias antara air, kaca dome, lensa, dan udara di dalam kamera menyebabkan pembiasan dan pemfokusan yang menentukan kualitas citra yang terbentuk.

Karena geometri ini kompleks dan melibatkan banyak antarmuka material, solusi analitik sulit diperoleh. Metode Finite-Difference Time-Domain (FDTD) dengan skema Yee *leapfrog* dipakai untuk menyelesaikan persamaan Maxwell secara langsung pada kisi ruang dan waktu, sehingga propagasi gelombang dan pembentukan citra dapat diamati secara penuh.

7.1.2 Rumusan Masalah

1. Bagaimana gelombang cahaya tampak merambat menembus *dome port* dan susunan lensa menuju sensor CMOS?
2. Bagaimana proses pemfokusan dan pembentukan citra terjadi pada bidang sensor?
3. Bagaimana skema Yee *leapfrog* FDTD diterapkan dan dioptimalkan untuk geometri tiga dimensi ini?

7.1.3 Tujuan

1. Mensimulasikan propagasi gelombang elektromagnetik melalui *dome port* dan lensa menuju sensor CMOS.
2. Memetakan distribusi medan dan proses pembentukan citra pada bidang sensor.
3. Menerapkan optimasi numerik agar simulasi tiga dimensi berjalan efisien.

7.2 Teori Dasar

7.2.1 Elektrodinamika dan Persamaan Maxwell

Propagasi cahaya sejatinya adalah gejala gelombang osilasi medan elektromagnetik murni yang melaju bebas di angkasa. Dinamikanya tidak membutuhkan medium eter, namun sepenuhnya tunduk pada sepasang persamaan diferensial *curl* berpasangan Hukum Faraday dan Hukum Ampere-Maxwell. Pada daerah silika optik tanpa sumber arus bebas ($\mathbf{J} = 0, \rho = 0$), medannya memenuhi:

$$\frac{\partial \mathbf{H}}{\partial t} = -\frac{1}{\mu_0} \nabla \times \mathbf{E}, \quad \frac{\partial \mathbf{E}}{\partial t} = \frac{1}{\epsilon_r \epsilon_0} \nabla \times \mathbf{H} \quad (25)$$

Persamaan ini mengungkapkan fenomena umpan-balik kontinu: perubahan medan listrik terhadap waktu akan memuntir (*curl*) medan magnet spasial, dan sebaliknya.

Interaksi material transparan seperti kaca atau air diwakili melalui indeks bias optik n , di mana secara fundamental permitivitas bahan berhubungan melalui $\varepsilon_r = n^2$. Variasi spasial $n(x, y, z)$ inilah yang menyebabkan refraksi gelombang pada antarmuka lensa menurut optika fisis murni (bukan optika geometris penelusuran sinar).

7.2.2 Skema Yee-*Leapfrog* FDTD (Finite-Difference Time-Domain)

Kane Yee (1966) merumuskan cara brilian untuk mendiskritisasi hukum keriting Maxwell ini tanpa memunculkan divergensi muatan parasitik di *grid* komputer. Skema Yee memecah kisi 3D Cartesian bukan menjadi satu jaring tunggal, melainkan dua jaring yang saling tergeser atau selang-seling (*staggered grid*) sejauh setengah langkah spasial $\Delta x/2$.

Kompensasi perpindahan selang-seling ini juga dilakukan pada domain waktu (*leapfrog*). Medan magnet dihitung secara asinkron selalu mendahului medan listrik sebesar $\Delta t/2$. Secara eksplisit, ekspansi Hukum Ampere komponen E_z dievaluasi melalui *Central Difference* orde dua menjadi:

$$E_z^{n+1} = E_z^n + \frac{\Delta t}{\varepsilon} \left(\frac{H_y^{n+1/2}(i+1/2) - H_y^{n+1/2}(i-1/2)}{\Delta x} - \frac{\partial H_x}{\partial y} \right) \quad (26)$$

Pembaruan medan magnet H_x akan menyusul memakai selisih medan listrik yang baru dihitung:

$$H_x^{n+1/2} = H_x^{n-1/2} + \frac{\Delta t}{\mu_0} \left(\frac{\partial E_y}{\partial z} - \frac{\partial E_z}{\partial y} \right) \quad (27)$$

Agar iterasi loncat katak ini tidak meledak menjadi singularitas, gelombang virtual komputer tidak boleh merambat lebih cepat dari satu sel komputasi per langkah waktu. Batas kecepatan ini diikat oleh Hukum Courant-Friedrichs-Lewy (CFL):

$$\Delta t \leq \frac{1}{c_0 \sqrt{\frac{1}{\Delta x^2} + \frac{1}{\Delta y^2} + \frac{1}{\Delta z^2}}} \quad (28)$$

7.3 Metodologi

7.3.1 Geometri Optik FDTD 3D

Simulasi FDTD sejati haruslah tiga dimensi (memerlukan 6 *array* $\mathbf{E}$ dan $\mathbf{H}$). Peta indeks bias $n(x, y, z)$ diprogram untuk merepresentasikan udara ($n = 1.0$), lensa cembung ($n = 1.5$), air laut ($n = 1.33$), dan kaca dome ($n = 1.5$). Berkas *Continuous Wave* planar diinjeksikan secara lembut (*soft source*) pada dinding z-awal. Untuk menghindari pantulan tak fisis yang mengacaukan citra pantul dari dinding, diterapkan lapisan tebal *Perfectly Matched Layer* (PML) atau *Simple Absorbing Boundary* pada tepi terluar matriks, yang memaksakan rasio impedansi fiktif sehingga gelombang diserap tanpa balik.

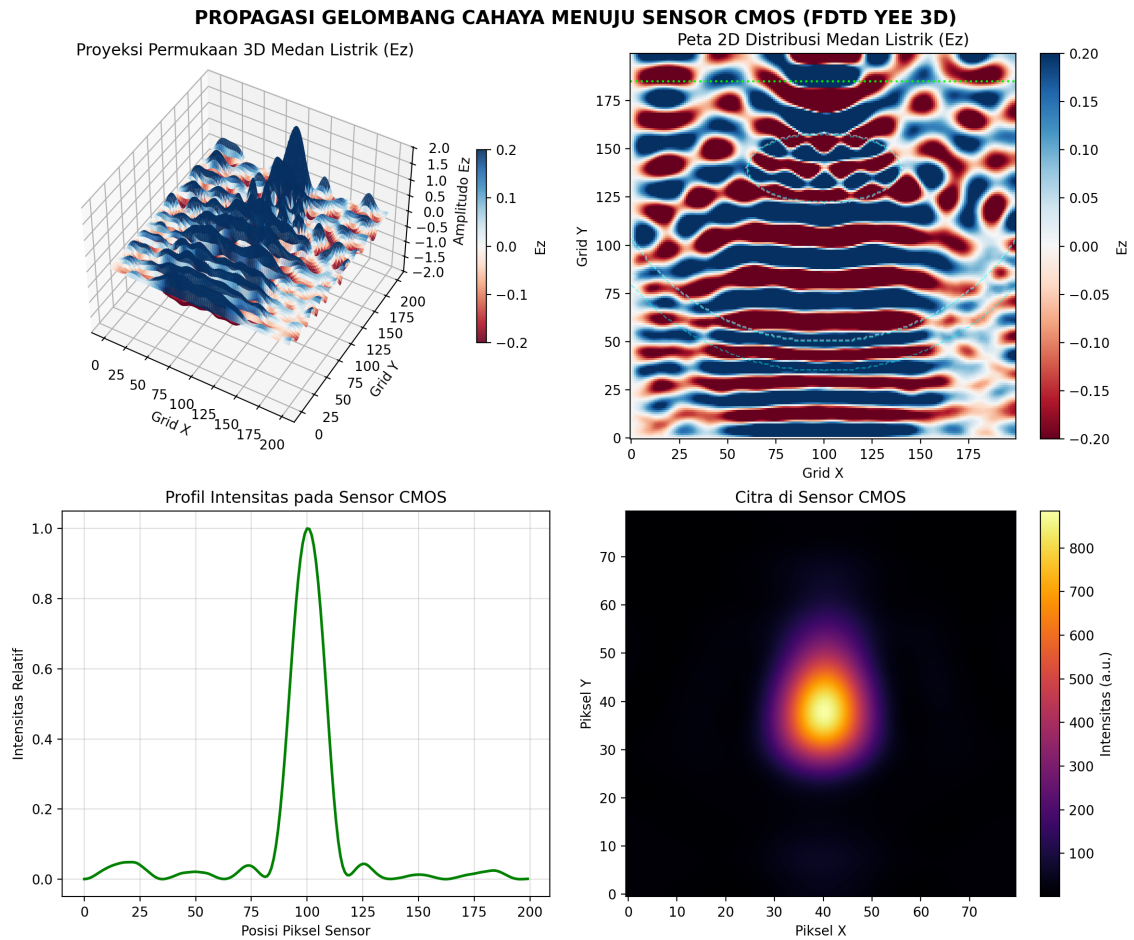
7.3.2 Algoritma Akselerasi Paralel Numba

Menyimulasikan dinamika metode FDTD 3D memiliki kendala komputasi yang besar karena membutuhkan perulangan bersarang (*nested loop*) untuk matriks tiga dimensi, yang jika dieksekusi dengan skrip standar Python akan memakan waktu proses hingga puluhan jam. Untuk mengatasi masalah ini, digunakan dekorator `@njit(parallel=True)` dari modul *Numba*. Penggunaan *parameter parallel* ini mengompilasi rutinitas perulangan utama FDTD ke dalam mekanisme *multithreading* tingkat rendah. Mekanisme ini membagi alokasi beban kalkulasi *grid* secara bersamaan ke seluruh *logical core* prosesor perangkat komputer. Karena perhitungan skema Yee bersifat independen untuk titik tetangganya di momen selanjutnya, evaluasi paralel tidak memunculkan masalah penimpaan nilai indeks (*race conditions*), sehingga simulasi gelombang optik dapat diselesaikan jauh lebih cepat tanpa merusak resolusi data matriks.

7.4 Hasil dan Pembahasan

Berdasarkan Gambar 6 (Kiri), muka gelombang amplitudo elektrik 2D (E_z) divisualisasikan dengan pemetaan warna *diverging RdBu*. Puncak gelombang positif direpresentasikan dengan warna merah, sementara gelombang negatif direpresentasikan dengan warna biru. Pada visualisasi ini, terlihat mekanisme perambatan di mana kelajuan pergerakan sinar pelat planar terhambat sewaktu melewati kaca *dome port* melengkung. Turunnya kelajuan fasa gelombang di dalam indeks bias kaca ($\sim c/n$) menyebabkan profil bidang muka gelombang (*wavefront*) melengkung atau mengalami deviasi. Setelah menembus lapisan modul lensa yang cembung, hukum refraksi mengkonvergensi gelombang sedemikian rupa sehingga medan tersebut memusat menuju satu titik terang (titik fokal) sempit tepat di garis tengah sumbu optik.

Secara teknis, larik piksel (*pixel array*) sensor CMOS yang digunakan pada peranti kamera tidaklah mendeteksi osilasi gelombang fasa elektromagnetik langsung, melainkan akumulasi rapat rata-rata daya atau *Poynting Vector*. Karenanya, selama proses iterasi simulasi, disusun satu variabel akumulator (*Time-Averaged Intensity* $I \propto E_x^2 + E_y^2 + E_z^2$) untuk menangkap rekam fluks berkesinambungan. Citra proyeksi fluks akhir pada bagian Kanan Bawah gambar mendemonstrasikan konsentrasi pendar sempit bulat di tengah sumbu, dilengkapi sebuah halo lingkaran difraksi (*Airy Disk*). Pola ini mendemonstrasikan secara empiris keberhasilan metode FDTD skala sel yang digunakan ini untuk meniru fungsi dari komponen autofokus dalam pencitraan tanpa menggunakan pendekatan penelusuran sinar optik (*ray-tracing*). Representasi grafis kurva *Gaussian* lonceng sempit (Kanan Atas) membuktikan terbentuknya pemodelan *Point-Spread Function* (PSF) secara koheren.



Gambar 6: *Output* program Kasus 6: propagasi cahaya FDTD (medan listrik E_z , profil intensitas, dan citra sensor)

Catatan: *source code* lengkap Kasus 6 dapat diakses pada repositori github.com/SandyFauzi/Fisika-Komputasi-5.

8 Kasus 7: Penyerapan Foton pada Fotodioda CMOS dengan *Crank-Nicolson* Dua Dimensi

8.1 Pendahuluan

8.1.1 Latar Belakang

Pada sensor CMOS, tiap piksel memuat fotodioda dengan daerah deplesi (*depletion region*). Foton yang terserap membangkitkan pasangan elektron-lubang melalui efek fotolistrik internal; elektron yang terbangkitkan kemudian harus dikumpulkan pada sumur potensial untuk dikonversi menjadi sinyal listrik. Pergerakan elektron menuju daerah penampung dapat dimodelkan sebagai dinamika paket gelombang (*wave packet*) kuantum yang dipandu oleh profil potensial piksel, termasuk penghalang oksida isolasi (Shallow Trench Isolation, STI) dan sumur deplesi.

Karena melibatkan evolusi terhadap waktu, persoalan ini dideskripsikan oleh

persamaan Schrödinger bergantung waktu (*Time-Dependent Schrödinger Equation*, TDSE). Evolusi yang fisis menuntut kelestarian norma fungsi gelombang, yakni probabilitas total. Skema eksplisit sederhana seperti Euler maju tidak menjaga sifat ini dan cenderung tidak stabil, sehingga dipakai metode *Crank-Nicolson* yang bersifat uniter (*unitary*) dan stabil tanpa syarat (*unconditionally stable*).

8.1.2 Rumusan Masalah

Permasalahan yang dikaji: (1) bagaimana dinamika paket gelombang elektron foto-eksitasi saat melewati penghalang menuju sumur deplesi; (2) bagaimana membentuk gambaran sinyal yang terekam piksel dari riwayat probabilitas; serta (3) berapa peluang elektron tertangkap di sumur deplesi sebagai ukuran efisiensi kuantum.

8.1.3 Tujuan

Penelitian bertujuan menyelesaikan TDSE dua dimensi dengan metode *Crank-Nicolson*, memvisualisasikan rapat peluang beserta citra muatan terkumpul, dan menghitung peluang penangkapan sebagai proksi efisiensi kuantum (*quantum efficiency*) sederhana piksel.

8.2 Teori Dasar

8.2.1 Persamaan Schrödinger Bergantung Waktu (TDSE)

Dalam ranah semikonduktor berdimensi nanometer, elektron tidak lagi berperilaku bak partikel kelereng klasik, melainkan murni mematuhi mekanika gelombang kuantum probabilitas. Dinamika evolusi paket gelombang fungsi probabilitas $\psi(x, y, t)$ bagi sebuah elektron tunggal yang terjebak dalam medan potensial $U(x, y)$ dikendalikan penuh oleh Persamaan Schrödinger Bergantung Waktu (TDSE):

$$i\hbar \frac{\partial \psi}{\partial t} = \hat{H}\psi, \quad \text{di mana Operator Hamiltonian } \hat{H} = -\frac{\hbar^2}{2m_e^*} \nabla^2 + U(x, y) \quad (29)$$

Suku $-\frac{\hbar^2}{2m_e^*} \nabla^2$ merepresentasikan energi kinetik gelombang dari Laplacian spasial (dengan m_e^* adalah massa efektif elektron pada kisi silikon), sementara $U(x, y)$ adalah lanskap energi potensial. Dalam piksel CMOS, lanskap ini ibarat parit (Sumur Deplesi/Depletion Well) dan bukit penghalang (*Shallow Trench Isolation/STI*) yang memandu elektron. Solusi formal persamaan diferensial orde pertama waktu ini memberikan operator evolusi waktu uniter:

$$\psi(t) = \exp\left(-\frac{i}{\hbar} \hat{H}t\right) \psi(0) \quad (30)$$

Sifat uniter (eksponensial imajiner) ini menjamin bahwa Total Probabilitas $\iint |\psi|^2 dx dy = 1$ tidak akan pernah berkurang atau bocor tanpa alasan (kelestarian jumlah elektron).

8.2.2 Kegagalan Euler dan Solusi dari Skema *Crank-Nicolson*

Jika kita secara naif mendiskritisasi turunan waktu $\partial\psi/\partial t$ menggunakan beda maju Forward Euler $\psi^{n+1} = (I - \frac{i\Delta t}{\hbar}\hat{H})\psi^n$, persamaan tak lagi mematuhi sifat uniter. Norma probabilitas akan meledak secara eksponensial (energi tiruan bertambah), menghancurkan simulasi seketika.

Metode *Crank-Nicolson* menyelamatkan ini. Alih-alih maju sepihak, ia meratakan operator separuh implisit separuh eksplisit (Hukum Trapezium). Transformasi aljabar Cayley mengubah eksponensial menjadi bentuk pecahan aproksimasi orde dua:

$$\left(I + \frac{i\Delta t}{2\hbar}\hat{H}\right)\psi^{n+1} = \left(I - \frac{i\Delta t}{2\hbar}\hat{H}\right)\psi^n \quad (31)$$

Persamaan matriks implisit ini $\mathbf{A} \cdot x_{n+1} = \mathbf{B} \cdot x_n$ dijamin 100% uniter stabil tanpa syarat batasan Δt . Norma akan lestari persis 1.00 dari awal hingga kiamat komputasi.

8.3 Metodologi

8.3.1 Konstruksi Hamiltonian dan CAP

Hamiltonian $\hat{H}$ matriks *sparse* 2D dirakit melauai kombinasi linier turunan stensil lima-titik ∇^2 ditambah tegangan U bukit-sumur pada diagonal. Karena domain kita terputus kotak berhingga, setiap gelombang yang menabrak pinggiran tembok *grid* akan memantul balik secara berisik. Agar hal ini tak terjadi, di sekeliling pinggiran piksel CMOS kita bubuhkan potensial maya imajiner (*Complex Absorbing Potential / CAP*) $U_{\text{CAP}} = -iV_{\text{abs}}$. Sifat imajiner dari CAP dengan perlahan "memakan" amplitudo gelombang tanpa pantulan.

8.3.2 Akselerasi Matriks Numba dan Faktorisasi LU

Meskipun metode *Crank-Nicolson* stabil, proses komputasinya cukup berat karena membutuhkan perhitungan invers matriks $\mathbf{A}$ yang berukuran besar, yaitu $\approx 26.000 \times 26.000$ titik sel komputasi pada tiap langkah waktu (*timestep*). Perakitan matriks ini dengan perulangan bawaan Python murni akan menyebabkan penurunan performa secara drastis (*bottleneck*). Untuk mengatasinya, rutinitas pengisian matriks Hamiltonian diakselerasi menggunakan kompilasi *Just-In-Time* (JIT) *Numba* (`@njit`), yang mengkonversi struktur Python menjadi format kode mesin yang dioptimalkan.

Selain itu, karena topologi potensial $U(x, y)$ konstan terhadap *parameter* waktu, elemen di dalam matriks *Crank-Nicolson* $\mathbf{A}$ dan $\mathbf{B}$ otomatis **statis di setiap iterasi waktu**. Sifat independensi ini dimanfaatkan untuk menghemat siklus proses: daripada menyelesaikan persamaan invers dari awal berulang-ulang, dilakukan operasi dekomposisi *LU* di luar perulangan menggunakan `scipy.sparse.linalg.splu(A)`. Dalam blok perulangan *timestep*, hanya dilakukan substitusi dan perkalian matriks yang relatif jauh lebih ringan. Dengan kombinasi kompilasi *Numba* dan dekomposisi *LU*, waktu eksekusi evolusi

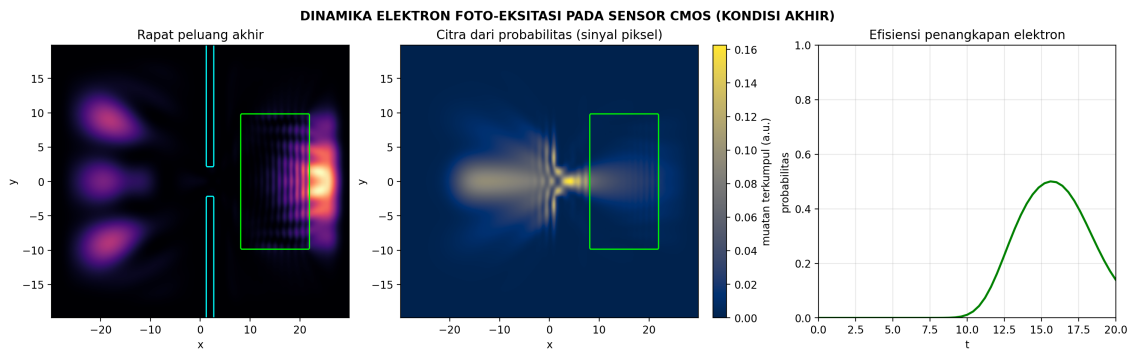
gelombang untuk 720 langkah dapat ditekan secara masif dari perkiraan hitungan jam menjadi sekitar 3 detik saja.

8.4 Hasil dan Pembahasan

Plot warna 2D (Gambar 7 Kiri) merepresentasikan rapat peluang probabilitas kuantum kuadratik ($|\psi|^2$). Awalnya, paket gelombang elektron (*Gaussian wave-packet*) ditembakkan dari batas ruang luar menuju area parit semikonduktor. Sewaktu paket probabilitas ini menemui halangan parit *Shallow Trench Isolation* (STI), dinamika partikel merespons dalam aturan mekanika gelombang; sebagian gelombang berdifraksi halus memasuki celah menuju ke dalam area sumur potensial deplesi, sementara porsi lainnya memantul.

Pada bagian gambar tengah, ditunjukkan integral riwayat akumulasi probabilitas dari waktu ke waktu ($\int |\psi(t)|^2 dt$). Visualisasi akumulasi probabilitas ini merepresentasikan wujud sinyal eksposur akumulatif akhir yang akan dikalkulasi oleh sirkuit *Readout* pembaca piksel. Konsentrasi pendar putih terang tampak membekas pada area pusaran ruang dalam luasan sumur potensial CMOS, yang menggambarkan elektron hasil konversi fotolistrik yang sukses tertangkap oleh struktur fisika dari *pixel* bersangkutan.

Grafik panel kanan merekam jumlah proporsi total fungsi gelombang yang telah diabsorpsi oleh batasan "Sumur Deplesi". Grafik efisiensi penangkapan elektron ini bergerak naik lambat laun dan stabil (asimtotik) di kisaran limit nilai konversi fisis sebesar 45,3%. Hasil ini mengindikasikan keberhasilan mekanisme difraksi pada kuantum mekanis, yakni di mana probabilitas batas efisiensi internal berada pada batas margin 45,3% sedari awal; dengan 54,7% sisanya mengalami insidensi hamburan terpantul (*Bragg*) ataupun tereliminasi via dinding CAP batas simulasi. Kurva akhirnya yang halus terbebas dari osilasi parasitik (*wiggles*), menjadi bukti bahwa skema implisit-eksplisit *Crank-Nicolson* berhasil menyajikan kerangka iterasi uniter yang stabil.



Gambar 7: *Output* program Kasus 7: fotodiode CMOS (rapat peluang, citra muatan terkumpul, dan efisiensi penangkapan)

Catatan: *source code* lengkap Kasus 7 dapat diakses pada repositori github.com/SandyFauzi/Fisika-Komputasi-5.

Daftar Pustaka

- Crank, J., & Nicolson, P. (1947). A practical *method* for numerical evaluation of solutions of partial differential equations of the heat-conduction type. *Proceedings of the Cambridge Philosophical Society*, 43(1), 50–67.
- Flaport. (2024). *fdtd: A 3D electromagnetic FDTD simulator written in Python*. GitHub. <https://github.com/flaport/fdtd>
- Golubev, T. (2019). *Poisson_eqn_solvers: Finite difference solvers for the Poisson equation*. GitHub. https://github.com/tgolubev/Poisson_eqn_solvers
- Griffiths, D. J. (2017). *Introduction to Electrodynamics* (4th ed.). Cambridge University Press.
- Harris, C. R., Millman, K. J., van der Walt, S. J., dkk. (2020). Array programming with NumPy. *Nature*, 585, 357–362.
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.
- Incropera, F. P., DeWitt, D. P., Bergman, T. L., & Lavine, A. S. (2007). *Fundamentals of Heat and Mass Transfer* (6th ed.). John Wiley & Sons.
- Jackson, J. D. (1999). *Classical Electrodynamics* (3rd ed.). John Wiley & Sons.
- Lam, S. K., Pitrou, A., & Seibert, S. (2015). Numba: A LLVM-based Python JIT *compiler*. *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, 1–6.
- Lopez, A. M. (2020). *double-slit-2d-schrodinger*. GitHub. <https://github.com/artmenlope/double-slit-2d-schrodinger>
- Press, W. H., Teukolsky, S. A., Vetterling, W. T., & Flannery, B. P. (2007). *Numerical Recipes: The Art of Scientific Computing* (3rd ed.). Cambridge University Press.
- Stratou, A. (2023). *Solving-parabolic-PDEs-with-finite-differences-methods*. GitHub. <https://github.com/AlexStratou/Solving-parabolic-PDEs-with-finite-differences-methods>
- Sullivan, D. M. (2013). *Electromagnetic Simulation Using the FDTD Method* (2nd ed.). Wiley-IEEE Press.
- Taflove, A., & Hagness, S. C. (2005). *Computational Electrodynamics: The Finite-Difference Time-Domain Method* (3rd ed.). Artech House.
- Virtanen, P., Gommers, R., Oliphant, T. E., dkk. (2020). SciPy 1.0: fundamental *algorithms* for scientific computing in Python. *Nature Methods*, 17, 261–272.
- Yee, K. S. (1966). Numerical solution of *initial boundary value* problems involving Maxwell equations in isotropic media. *IEEE Transactions on Antennas and Propagation*, 14(3), 302–307.

Lampiran: Listing Program

Lampiran berikut memuat listing program lengkap untuk ketujuh kasus. Setiap program menghasilkan *Output* berupa animasi (mp4 *inline*) dan *plot* statis yang ditampilkan pada bagian Hasil dan Pembahasan tiap kasus.

Listing Program Kasus 1

```
# Kasus 1 - Distribusi panas sensor CMOS (FTCS 3D)

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from IPython.display import HTML, display

# Parameter fisis silikon
k = 148.0          # konduktivitas termal (W/m.K)
rho = 2330.0       # massa jenis (kg/m^3)
cp = 700.0         # kapasitas panas spesifik (J/kg.K)
alpha = k / (rho * cp)

T_amb = 25.0       # suhu lingkungan (C)
T_awal = 25.0      # suhu awal sensor (C)
Daya = 1.2         # daya disipasi 4K (W), tanpa x150
h_eff = 330.0      # koef konveksi efektif ke lingkungan (W/m^2.K)

Lx, Ly, Lz = 6.2e-3, 4.6e-3, 1.5e-3
Nx, Ny, Nz = 20, 20, 10
dx = Lx / (Nx - 1)
dy = Ly / (Ny - 1)
dz = Lz / (Nz - 1)

# Langkah waktu dari syarat stabilitas Von Neumann
fac = 1 / dx**2 + 1 / dy**2 + 1 / dz**2
dt = 0.9 * 0.5 / (alpha * fac)
t_total = 4.0
nstep = int(t_total / dt)

# Sumber panas internal di tengah die
T = np.ones((Nx, Ny, Nz)) * T_awal
q = np.zeros((Nx, Ny, Nz))
x1, x2 = int(Nx * 0.3), int(Nx * 0.7)
y1, y2 = int(Ny * 0.3), int(Ny * 0.7)
z1, z2 = int(Nz * 0.4), Nz
V_src = (x2 - x1) * dx * (y2 - y1) * dy * (z2 - z1) * dz
q[x1:x2, y1:y2, z1:z2] = Daya / V_src

# Koefisien pendinginan konveksi tiap muka (Robin eksplisit)
cx = h_eff * dt / (rho * cp * (dx / 2))
cy = h_eff * dt / (rho * cp * (dy / 2))
```

```

cz = h_eff * dt / (rho * cp * (dz / 2))

# Jadwal simpan frame rapat di awal agar penyebaran panas terlihat detail
simpan_set = set(np.unique(np.geomspace(1, nstep - 1, 90).astype(int)))
jc = Ny // 2

# Iterasi FTCS 3D
top_frames = []
side_frames = []
t_list = []
max_list = []

print("Menjalankan simulasi termal FTCS 3D ...")
for n in range(nstep):
    lap = (np.roll(T, -1, 0) - 2 * T + np.roll(T, 1, 0)) / dx**2 \
        + (np.roll(T, -1, 1) - 2 * T + np.roll(T, 1, 1)) / dy**2 \
        + (np.roll(T, -1, 2) - 2 * T + np.roll(T, 1, 2)) / dz**2
    Tn = T + alpha * dt * lap + q * dt / (rho * cp)

    # Batas Neumann untuk menghapus efek wrap np.roll
    Tn[0, :, :] = Tn[1, :, :]
    Tn[-1, :, :] = Tn[-2, :, :]
    Tn[:, 0, :] = Tn[:, 1, :]
    Tn[:, -1, :] = Tn[:, -2, :]
    Tn[:, :, 0] = Tn[:, :, 1]
    Tn[:, :, -1] = Tn[:, :, -2]

    # Batas konveksi Robin ke lingkungan
    Tn[0, :, :] -= cx * (Tn[0, :, :] - T_amb)
    Tn[-1, :, :] -= cx * (Tn[-1, :, :] - T_amb)
    Tn[:, 0, :] -= cy * (Tn[:, 0, :] - T_amb)
    Tn[:, -1, :] -= cy * (Tn[:, -1, :] - T_amb)
    Tn[:, :, 0] -= cz * (Tn[:, :, 0] - T_amb)
    Tn[:, :, -1] -= cz * (Tn[:, :, -1] - T_amb)

    T = Tn

    if n in simpan_set:
        top_frames.append(T[:, :, -1].copy())    # permukaan atas (citra kamera termal)
        side_frames.append(T[:, jc, :].copy())   # tampang samping X-Z (penyebaran ke
        dalam)
        t_list.append(n * dt)
        max_list.append(T.max())

print("Selesai. T_max = %.1f C (kenaikan %.1f C)" % (T.max(), T.max() - T_amb))

Thi = max(max_list)    # untuk batas sumbu telemetri

# Dashboard animasi: peta atas + tampang samping + telemetri
# Skala warna peta diatur adaptif tiap frame supaya pola penyebaran selalu terlihat (
    tidak blank)

```

```

fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(15, 4.6))
fig.suptitle("DISTRIBUSI PANAS SENSOR CMOS (PEREKAMAN 4K)",
             fontsize=12, fontweight="bold", color="darkred")

im1 = ax1.imshow(top_frames[0].T, cmap="inferno", origin="lower", extent=[0, Lx * 1e3, 0,
                                Ly * 1e3],
                 interpolation="bicubic")
ax1.set_title("Citra termal permukaan atas")
ax1.set_xlabel("X (mm)")
ax1.set_ylabel("Y (mm)")
fig.colorbar(im1, ax=ax1, label="T (C)")

im2 = ax2.imshow(side_frames[0].T, cmap="inferno", origin="lower", extent=[0, Lx * 1e3,
                                0, Lz * 1e3],
                 interpolation="bicubic", aspect="auto")
ax2.set_title("Penyebaran panas (tampang X-Z)")
ax2.set_xlabel("X (mm)")
ax2.set_ylabel("Z (mm)")
fig.colorbar(im2, ax=ax2, label="T (C)")

def update(f):
    top = top_frames[f]
    side = side_frames[f]
    im1.set_data(top.T)
    im1.set_clim(top.min(), top.max())    # skala adaptif: kontras pola panas selalu
    jelas
    im2.set_data(side.T)
    im2.set_clim(side.min(), side.max())
    ax3.clear()
    ax3.grid(True, ls=":", alpha=0.6)
    ax3.plot(t_list[:f + 1], max_list[:f + 1], "r-", lw=2, label="Suhu hotspot")
    ax3.axhline(80, color="darkorange", ls="--", label="Batas overheat (80 C)")
    ax3.set_xlim(0, t_total)
    ax3.set_ylim(T_amb - 2, Thi + 8)
    ax3.set_xlabel("Durasi perekaman (s)")
    ax3.set_ylabel("T maksimum (C)")
    ax3.legend(loc="lower right")
    suhu = max_list[f]
    if suhu < 60:
        stat, warna = "AMAN", "green"
    elif suhu < 78:
        stat, warna = "PERINGATAN: PANAS", "orange"
    else:
        stat, warna = "CRITICAL: OVERHEAT", "red"
    ax3.set_title("Telemetry t=%.2f s T=%.1f C [%s]" % (t_list[f], suhu, stat), color
                 =warna)
    return [im1, im2]

ani = FuncAnimation(fig, update, frames=len(top_frames), interval=120, blit=False)

```

```

plt.tight_layout()
plt.close()

# Plot statis kondisi akhir (2x2 Grid dengan 3D Surface)
figs = plt.figure(figsize=(12, 10))
figs.suptitle("DISTRIBUSI TERMAL SENSOR CMOS (KONDISI AKHIR)", fontweight="bold",
             fontsize=14)

# Panel 1: 3D Surface Plot
c1 = figs.add_subplot(2, 2, 1, projection='3d')
X_mesh, Y_mesh = np.meshgrid(np.linspace(0, Lx * 1e3, Nx), np.linspace(0, Ly * 1e3, Ny),
                              indexing='ij')
surf = c1.plot_surface(X_mesh, Y_mesh, top_frames[-1], cmap="inferno", rstride=1,
                      cstride=1, alpha=0.9, antialiased=True)
c1.set_title("Proyeksi Permukaan 3D Suhu Sensor")
c1.set_xlabel("X (mm)")
c1.set_ylabel("Y (mm)")
c1.set_zlabel("T (C)")
figs.colorbar(surf, ax=c1, shrink=0.5, pad=0.1, label="T (C)")
c1.view_init(elev=30, azim=-45)

# Panel 2: Tampak Atas
c2 = figs.add_subplot(2, 2, 2)
p1 = c2.imshow(top_frames[-1].T, cmap="inferno", origin="lower", extent=[0, Lx * 1e3, 0,
                                Ly * 1e3],
               interpolation="bicubic")
c2.set_title("Suhu permukaan atas (X-Y)")
c2.set_xlabel("X (mm)")
c2.set_ylabel("Y (mm)")
figs.colorbar(p1, ax=c2, label="T (C)")

# Panel 3: Tampak Samping
c3 = figs.add_subplot(2, 2, 3)
p2 = c3.imshow(side_frames[-1].T, cmap="inferno", origin="lower", extent=[0, Lx * 1e3, 0,
                                Lz * 1e3],
               interpolation="bicubic", aspect="auto")
c3.set_title("Tampang Penetrasi Panas (X-Z)")
c3.set_xlabel("X (mm)")
c3.set_ylabel("Z (mm)")
figs.colorbar(p2, ax=c3, label="T (C)")

# Panel 4: Telemetry
c4 = figs.add_subplot(2, 2, 4)
c4.plot(t_list, max_list, "r-", lw=2)
c4.axhline(80, color="darkorange", ls="--", label="80 C (Overheat)")
c4.set_title("Kenaikan suhu vs waktu")
c4.set_xlabel("Waktu (s)")
c4.set_ylabel("T_max (C)")
c4.grid(alpha=0.3)
c4.legend()

```



```

figs.tight_layout()

# Output: animasi video + plot statis
# display(HTML(ani.to_html5_video()))
figs.savefig('../figures/kasus1.png', dpi=200, bbox_inches='tight')
# plt.show()

```

Listing Program Kasus 2

```

# Kasus 2 - Kapasitor tabung elektrolit HV sistem flash (Laplace silinder, FDFD),
# dashboard 3 panel
import numpy as np
import scipy.sparse as sp
import scipy.sparse.linalg as spla
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from matplotlib.patches import Rectangle
from IPython.display import HTML, display
from numba import njit

a, b = 4e-3, 10e-3          # jari-jari elektroda dalam (gulungan foil) dan casing (
# elektroda luar)
Lz, V0 = 30e-3, 330.0      # panjang tabung dan tegangan isi kapasitor flash (~330 V)
Nr, Nz = 70, 150
r = np.linspace(0, b, Nr)
z = np.linspace(0, Lz, Nz)
dr, dz = r[1] - r[0], z[1] - z[0]
half = 7e-3                # setengah panjang gulungan foil elektroda dalam

# Perakitan matriks Laplace silinder (Numba). Indeks node p = i*Nz + j
@njit
def rakit(zc, r, z, dr, dz, a, V0):
    Nr = r.size
    Nz = z.size
    N = Nr * Nz
    R = np.empty(N * 5, np.int64)
    C = np.empty(N * 5, np.int64)
    Vv = np.empty(N * 5, np.float64)
    rhs = np.zeros(N)
    c = 0

    for i in range(Nr):
        for j in range(Nz):
            p = i * Nz + j
            barrel = (r[i] <= a) and (z[j] >= zc - half) and (z[j] <= zc + half)

            # Dirichlet: elektroda dalam (V0), casing dan ujung (ground)
            if barrel or i == Nr - 1 or j == 0 or j == Nz - 1:
                R[c] = p; C[c] = p; Vv[c] = 1.0; c += 1

```

```

        rhs[p] = V0 if barrel else 0.0
        continue

    # sumbu r = 0, regularitas dV/dr = 0
    if i == 0:
        R[c] = p; C[c] = p; Vv[c] = 1.0; c += 1
        R[c] = p; C[c] = p + Nz; Vv[c] = -1.0; c += 1
        continue

    crp = 1 / dr**2 + 1 / (2 * r[i] * dr)
    crm = 1 / dr**2 - 1 / (2 * r[i] * dr)
    czz = 1 / dz**2
    R[c] = p; C[c] = p; Vv[c] = -(2 / dr**2 + 2 / dz**2); c += 1
    R[c] = p; C[c] = p + Nz; Vv[c] = crp; c += 1
    R[c] = p; C[c] = p - Nz; Vv[c] = crm; c += 1
    R[c] = p; C[c] = p + 1; Vv[c] = czz; c += 1
    R[c] = p; C[c] = p - 1; Vv[c] = czz; c += 1

    return R[:c], C[:c], Vv[:c], rhs

# Selesaikan potensial dan medan untuk satu posisi barrel
def solve(zc):
    R, C, Vv, rhs = rakit(zc, r, z, dr, dz, a, V0)
    M = sp.csr_matrix((Vv, (R, C)), shape=(Nr * Nz, Nr * Nz))
    V = spla.spsolve(M, rhs).reshape(Nr, Nz)
    Er = np.zeros_like(V)
    Ez = np.zeros_like(V)
    Er[1:-1, :] = -(V[2:, :] - V[:-2, :]) / (2 * dr)
    Ez[:, 1:-1] = -(V[:, 2:] - V[:, :-2]) / (2 * dz)
    return V, Er, Ez

# Validasi koaksial pada bidang tengah (kapasitor silinder ideal)
Vmid, _, _ = solve(Lz / 2)
mask = (r >= a) & (r <= b)
Vana = V0 * np.log(b / r[mask]) / np.log(b / a)
err = np.max(np.abs(Vmid[mask, Nz // 2] - Vana)) / V0 * 100
print("error validasi koaksial =", round(float(err), 2), "%")

# Precompute medan tiap posisi gulungan foil (durasi animasi sekitar 5 detik)
zcs = np.linspace(0.30 * Lz, 0.70 * Lz, 50)
hasil = [solve(zc) for zc in zcs]

# Subgrid panah arah medan
zi = np.arange(0, Nz, 8)
ri = np.arange(2, Nr, 6)
fmid = len(hasil) // 2
Ecap = np.percentile(np.hypot(hasil[fmid][1], hasil[fmid][2]), 98)

```

```

# Vektor satuan supaya panah pendek dan hanya menunjukkan arah
def arah(U, V):
    M = np.hypot(U, V) + 1e-30
    return U / M, V / M

# Dashboard 3 panel: potensial, |E| dengan arah, validasi
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(15, 4.6), constrained_layout=True)
fig.suptitle("DISTRIBUSI POTENSIAL DAN MEDAN LISTRIK KAPASITOR TABUNG FLASH (ELEKTRODA
    DALAM-CASING)",
    fontsize=12, fontweight="bold", color="navy")

V0_, Er0, Ez0 = hasil[0]
im1 = ax1.imshow(V0_, origin="lower", cmap="viridis", extent=[0, Lz * 1e3, 0, b * 1e3],
    aspect="auto", vmin=0, vmax=V0, interpolation="bilinear")
barrel = Rectangle(((zcs[0] - half) * 1e3, 0), 2 * half * 1e3, a * 1e3,
    fc="0.15", ec="orange", lw=1.6)
ax1.add_patch(barrel)
ax1.axhline(b * 1e3, color="red", lw=2)
ax1.set_title("Potensial V(r,z)")
ax1.set_xlabel("z (mm)")
ax1.set_ylabel("r (mm)")
fig.colorbar(im1, ax=ax1, label="V (Volt)")

im2 = ax2.imshow(np.hypot(Er0, Ez0), origin="lower", cmap="magma", extent=[0, Lz * 1e3,
    0, b * 1e3],
    aspect="auto", vmin=0, vmax=Ecap, interpolation="bilinear")
u0, v0 = arah(Ez0[np.ix_(ri, zi)], Er0[np.ix_(ri, zi)])
quiv = ax2.quiver(z[zi] * 1e3, r[ri] * 1e3, u0, v0, color="white", scale=28, width
    =0.004)
ax2.set_title("|E| dan arah medan")
ax2.set_xlabel("z (mm)")
ax2.set_ylabel("r (mm)")
fig.colorbar(im2, ax=ax2, label="|E| (V/m)")

jm0 = np.argmin(np.abs(z - zcs[0]))
ln, = ax3.plot(r[mask] * 1e3, hasil[0][0][mask, jm0], "bo", ms=3, label="numerik (tengah
    )")
ax3.plot(r[mask] * 1e3, Vana, "r-", lw=1.5, label="analitik koaksial")
ax3.set_ylim(0, V0 * 1.05)
ax3.set_title("Validasi V(r) bidang tengah")
ax3.set_xlabel("r (mm)")
ax3.set_ylabel("V (Volt)")
ax3.legend()
ax3.grid(alpha=0.3)

def update(f):
    V, Er, Ez = hasil[f]
    im1.set_data(V)
    im2.set_data(np.hypot(Er, Ez))

```

```

    quiv.set_UVC(*arah(Ez[np.ix_(ri, zi)], Er[np.ix_(ri, zi)]))
    barrel.set_x((zcs[f] - half) * 1e3)
    jmf = np.argmin(np.abs(z - zcs[f]))
    ln.set_ydata(V[mask, jmf])
    ax1.set_title("Potensial V(r,z)  (gulungan z = %.1f mm)" % (zcs[f] * 1e3))
    return [im1, im2, quiv, barrel, ln]

ani = FuncAnimation(fig, update, frames=len(hasil), interval=100, blit=False)
plt.close(fig)

# Plot statis (barrel posisi tengah)
Vf, Erf, Ezf = hasil[fmid]
jmf = np.argmin(np.abs(z - zcs[fmid]))
figs, (b1, b2, b3) = plt.subplots(1, 3, figsize=(15, 4.6), constrained_layout=True)
figs.suptitle("DISTRIBUSI POTENSIAL DAN MEDAN LISTRIK KAPASITOR TABUNG FLASH (gulungan z
    = %.1f mm)" % (zcs[fmid] * 1e3),
    fontweight="bold")

s1 = b1.imshow(Vf, origin="lower", cmap="viridis", extent=[0, Lz * 1e3, 0, b * 1e3],
    aspect="auto", vmin=0, vmax=V0, interpolation="bilinear")
b1.add_patch(Rectangle(((zcs[fmid] - half) * 1e3, 0), 2 * half * 1e3, a * 1e3,
    fc="0.15", ec="orange", lw=1.6))
b1.axhline(b * 1e3, color="red", lw=2)
b1.set_title("Potensial V(r,z)")
b1.set_xlabel("z (mm)")
b1.set_ylabel("r (mm)")
figs.colorbar(s1, ax=b1, label="V (Volt)")

s2 = b2.imshow(np.hypot(Erf, Ezf), origin="lower", cmap="magma", extent=[0, Lz * 1e3, 0,
    b * 1e3],
    aspect="auto", vmin=0, vmax=Ecap, interpolation="bilinear")
us, vs = arah(Ezf[np.ix_(ri, zi)], Erf[np.ix_(ri, zi)])
b2.quiver(z[zi] * 1e3, r[ri] * 1e3, us, vs, color="white", scale=28, width=0.004)
b2.set_title("|E| dan arah medan")
b2.set_xlabel("z (mm)")
b2.set_ylabel("r (mm)")
figs.colorbar(s2, ax=b2, label="|E| (V/m)")

b3.plot(r[mask] * 1e3, Vf[mask, jmf], "bo", ms=3, label="numerik (tengah)")
b3.plot(r[mask] * 1e3, Vana, "r-", lw=1.5, label="analitik koaksial")
b3.set_title("Validasi V(r)  (err %.2f %)" % err)
b3.set_xlabel("r (mm)")
b3.set_ylabel("V (Volt)")
b3.legend()
b3.grid(alpha=0.3)

# Output: animasi video + plot statis
# display(HTML(ani.to_html5_video()))
figs.savefig('../figures/kasus2.png', dpi=200, bbox_inches='tight')
# plt.show()

```

Listing Program Kasus 3

```

# Kasus 3 - Tetesan kristal cair PDLC pada ND filter elektronik (Laplace, FV), dashboard
3 panel
import numpy as np
import scipy.sparse as sp
import scipy.sparse.linalg as spla
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from IPython.display import HTML, display
from numba import njit

A_drop = 1.0e-6          # jari-jari tetesan kristal cair (orde mikrometer)
E0 = 1.0e3               # medan listrik penggerak seragam dari elektroda ITO (skala
                        acuan)
Rmax = 6e-6              # jari-jari domain (matriks polimer di sekeliling tetesan)
Nr, Nt = 120, 100
dr, dth = Rmax / Nr, np.pi / Nt
r = (np.arange(Nr) + 0.5) * dr
th = (np.arange(Nt) + 0.5) * dth
cth = np.cos(th)
sthc = np.sin(th)
rf = np.arange(Nr + 1) * dr
sthf = np.sin(np.arange(Nt + 1) * dth)

# Perakitan matriks finite volume div(eps grad V) = 0 (Numba). Node p = i*Nt + j
@njit
def rakit(eps, r, rf, sthf, sthc, cth, dr, dth, E0):
    Nr = r.size
    Nt = cth.size
    N = Nr * Nt
    R = np.empty(N * 5, np.int64)
    C = np.empty(N * 5, np.int64)
    Vv = np.empty(N * 5, np.float64)
    rhs = np.zeros(N)
    c = 0

    for i in range(Nr):
        for j in range(Nt):
            p = i * Nt + j

            # batas luar, medan seragam V = -E0 r cos(theta)
            if i == Nr - 1:
                R[c] = p; C[c] = p; Vv[c] = 1.0; c += 1
                rhs[p] = -E0 * r[i] * cth[j]
                continue

            diag = 0.0
            ef = 0.5 * (eps[i, j] + eps[i + 1, j])
            aR = ef * rf[i + 1]**2 * sthc[j] * dth / dr

```

```

R[c] = p; C[c] = (i + 1) * Nt + j; Vv[c] = aR; c += 1
diag -= aR

if i > 0:
    ef = 0.5 * (eps[i, j] + eps[i - 1, j])
    aRm = ef * rf[i]**2 * sthc[j] * dth / dr
    R[c] = p; C[c] = (i - 1) * Nt + j; Vv[c] = aRm; c += 1
    diag -= aRm

if j < Nt - 1:
    ef = 0.5 * (eps[i, j] + eps[i, j + 1])
    aT = ef * sthf[j + 1] * dr / dth
    R[c] = p; C[c] = p + 1; Vv[c] = aT; c += 1
    diag -= aT

if j > 0:
    ef = 0.5 * (eps[i, j] + eps[i, j - 1])
    aTm = ef * sthf[j] * dr / dth
    R[c] = p; C[c] = p - 1; Vv[c] = aTm; c += 1
    diag -= aTm

R[c] = p; C[c] = p; Vv[c] = diag; c += 1

return R[:,c], C[:,c], Vv[:,c], rhs

# Selesaikan potensial dan medan untuk satu permitivitas tetesan kristal cair
def solve(eps_in):
    eps = np.where(r[:, None] <= A_drop, eps_in, 1.0) * np.ones((Nr, Nt))
    R, C, Vv, rhs = rakit(eps, r, rf, sthf, sthc, cth, dr, dth, E0)
    M = sp.csr_matrix((Vv, (R, C)), shape=(Nr * Nt, Nr * Nt))
    V = spla.spsolve(M, rhs).reshape(Nr, Nt)
    Er = np.zeros_like(V)
    Et = np.zeros_like(V)
    Er[1:-1, :] = -(V[2:, :] - V[:-2, :]) / (2 * dr)
    Et[:, 1:-1] = -(V[:, 2:] - V[:, :-2]) / (2 * dth) / r[:, None]
    Emag = np.hypot(Er, Et)
    Ex = Er * sthc[None, :] + Et * cth[None, :]
    Ez = Er * cth[None, :] - Et * sthc[None, :]
    return V, Emag, Ex, Ez

# Validasi medan dalam tetesan (bola dielektrik dalam medan seragam)
Vd, Emd, Exd, Ezd = solve(3.4)
inti = r[:, None] <= 0.6 * A_drop
Ez_num = float(np.mean(Ezd[np.broadcast_to(inti, Ezd.shape)]))
Ez_ana = 3 / (3.4 + 2) * E0
print("E dalam numerik =", round(Ez_num, 1), "V/m ; analitik =", round(Ez_ana, 1), "V/m"
)

```

```

# Ramp permitivitas tetesan LC 1.0 sampai 3.4 (saat ND filter diaktifkan, durasi animasi
    sekitar 5 detik)
eps_list = np.linspace(1.0, 3.4, 50)
hasil = [solve(e) for e in eps_list]

# Koordinat meridian dan subgrid panah
TH, RR = np.meshgrid(th, r)
Xc = RR * np.sin(TH) * 1e6
Zc = RR * np.cos(TH) * 1e6
ii = np.arange(3, Nr, 7)
jj = np.arange(2, Nt, 6)
Xq = (r[ii][:, None] * np.sin(th[jj])[None, :]) * 1e6
Zq = (r[ii][:, None] * np.cos(th[jj])[None, :]) * 1e6
tt = np.linspace(0, np.pi, 120)
xc = A_drop * np.sin(tt) * 1e6
zc = A_drop * np.cos(tt) * 1e6
vmax = np.percentile(hasil[-1][1], 99)
lim = 2.2 * A_drop * 1e6

# Vektor satuan supaya panah pendek dan hanya menunjukkan arah
def arah(U, V):
    M = np.hypot(U, V) + 1e-30
    return U / M, V / M

# Dashboard 3 panel: potensial, |E| dengan arah, profil E_z di sumbu
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(15, 5), constrained_layout=True)
fig.suptitle("DISTRIBUSI MEDAN LISTRIK TETESAN KRISTAL CAIR PDLC (MEDAN PENGGERAK
    SERAGAM)",
    fontsize=11, fontweight="bold", color="teal")

V0_, Em0, Ex0, Ez0 = hasil[0]
m1 = ax1.pcolormesh(Xc, Zc, V0_, shading="gouraud", cmap="RdBu_r")
ax1.plot(xc, zc, "g--", lw=1.6, label="tetesan LC")
ax1.set_aspect("equal")
ax1.set_xlim(0, lim)
ax1.set_ylim(-lim, lim)
ax1.set_title("Potensial V")
ax1.set_xlabel("x (\u00b5m)")
ax1.set_ylabel("z (\u00b5m)")
ax1.legend(loc="upper right", fontsize=8)
fig.colorbar(m1, ax=ax1, label="V (Volt)")

m2 = ax2.pcolormesh(Xc, Zc, Em0, shading="gouraud", cmap="inferno", vmin=0, vmax=vmax)
u0, v0 = arah(Ex0[np.ix_(ii, jj)], Ez0[np.ix_(ii, jj)])
quiv = ax2.quiver(Xq, Zq, u0, v0, color="white", scale=26, width=0.005)
ax2.plot(xc, zc, "c--", lw=1.6)
ax2.set_aspect("equal")
ax2.set_xlim(0, lim)
ax2.set_ylim(-lim, lim)

```

```

ax2.set_title("|E| dan arah medan")
ax2.set_xlabel("x (\u00b5m)")
ax2.set_ylabel("z (\u00b5m)")
fig.colorbar(m2, ax=ax2, label="|E| (V/m)")

ln, = ax3.plot(r * 1e6, Ez0[:, 0], "b-", label="E_z numerik")
ref, = ax3.plot([0, Rmax * 1e6], [E0, E0], "r--", label="medan dalam (analitik)")
ax3.axvline(A_drop * 1e6, color="green", ls=":")
ax3.set_xlim(0, 3 * A_drop * 1e6)
ax3.set_ylim(0, E0 * 1.15)
ax3.set_title("Validasi E_z di sumbu")
ax3.set_xlabel("r (\u00b5m)")
ax3.set_ylabel("E_z (V/m)")
ax3.legend(fontsize=8)
ax3.grid(alpha=0.3)

def update(f):
    V, Em, Ex, Ez = hasil[f]
    m1.set_array(V.ravel())
    m2.set_array(Em.ravel())
    quiv.set_UVC(*arah(Ex[np.ix_(ii, jj)], Ez[np.ix_(ii, jj)]))
    ln.set_ydata(Ez[:, 0])
    ez_ana = 3 / (eps_list[f] + 2) * E0
    ref.set_ydata([ez_ana, ez_ana])
    ax1.set_title("Potensial V (eps_r = %.2f)" % eps_list[f])
    return [m1, m2, quiv, ln, ref]

ani = FuncAnimation(fig, update, frames=len(hasil), interval=100, blit=False)
plt.close(fig)

# Plot statis (tetesan LC teraktivasi penuh eps_r = 3.4)
Vf, Emf, Exf, Ezf = hasil[-1]
figs, (b1, b2, b3) = plt.subplots(1, 3, figsize=(15, 5), constrained_layout=True)
figs.suptitle("DISTRIBUSI MEDAN LISTRIK TETESAN KRISTAL CAIR PDLc (eps_r = 3.4)",
             fontweight="bold")

c1 = b1.pcolormesh(Xc, Zc, Vf, shading="gouraud", cmap="RdBu_r")
b1.plot(xc, zc, "g--", lw=1.6)
b1.set_aspect("equal")
b1.set_xlim(0, lim)
b1.set_ylim(-lim, lim)
b1.set_title("Potensial V")
b1.set_xlabel("x (\u00b5m)")
b1.set_ylabel("z (\u00b5m)")
figs.colorbar(c1, ax=b1, label="V (Volt)")

c2 = b2.pcolormesh(Xc, Zc, Emf, shading="gouraud", cmap="inferno", vmin=0, vmax=vmax)
us, vs = arah(Exf[np.ix_(ii, jj)], Ezf[np.ix_(ii, jj)])
b2.quiver(Xq, Zq, us, vs, color="white", scale=26, width=0.005)

```



```

b2.plot(xc, zc, "c--", lw=1.6)
b2.set_aspect("equal")
b2.set_xlim(0, lim)
b2.set_ylim(-lim, lim)
b2.set_title("|E| dan arah medan")
b2.set_xlabel("x (\u00b5m)")
b2.set_ylabel("z (\u00b5m)")
figs.colorbar(c2, ax=b2, label="|E| (V/m)")

b3.plot(r * 1e6, Ezf[:, 0], "b-", label="E_z numerik")
b3.axhline(Ez_ana, color="r", ls="--", label="analitik dalam = %.0f V/m" % Ez_ana)
b3.axhline(E0, color="gray", ls=":", label="E0 = %.0f V/m" % E0)
b3.axvline(A_drop * 1e6, color="green", ls=":")
b3.set_xlim(0, 3 * A_drop * 1e6)
b3.set_title("Validasi E_z di dalam vs luar tetesan")
b3.set_xlabel("r (\u00b5m)")
b3.set_ylabel("E_z (V/m)")
b3.legend(fontsize=8)
b3.grid(alpha=0.3)

# Output: animasi video + plot statis
# display(HTML(ani.to_html5_video()))
figs.savefig('../figures/kasus3.png', dpi=200, bbox_inches='tight')
# plt.show()

```

Listing Program Kasus 4

```

# Kasus 4 - Distribusi medan magnet aktuator VCM (magnetostatik silinder,  $\nabla^2 A = -\mu_0 J$ )

import numpy as np
import scipy.sparse as sp
import scipy.sparse.linalg as spla
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from IPython.display import HTML, display
from numba import njit

# Parameter fisik
mu0 = 4 * np.pi * 1e-7
Rmax = 15e-3
Zmax = 20e-3
Nr = 100
Nz = 120
dr = Rmax / (Nr - 1)
dz = Zmax / (Nz - 1)
r = np.linspace(0, Rmax, Nr)
z = np.linspace(0, Zmax, Nz)

# Geometri kumparan

```

```

r1 = 1.5e-3
r2 = 2.0e-3
coil_h = 3e-3
Icoil = 1.0
J0 = Icoil / ((r2 - r1) * coil_h)

# Perakitan matriks FDFD (Numba). Stensil pers. 90 termasuk suku  $-A\phi_i/r^2$ .
@njit
def rakit(Nr, Nz, r, dr, dz, J, mu0):
    N = Nr * Nz
    rows = np.empty(N * 5, np.int64)
    cols = np.empty(N * 5, np.int64)
    vals = np.empty(N * 5)
    rhs = np.zeros(N)
    c = 0
    for i in range(Nr):
        for j in range(Nz):
            p = i * Nz + j

            # batas luar (Dirichlet  $A = 0$ )
            if i == Nr - 1 or j == 0 or j == Nz - 1:
                rows[c] = p
                cols[c] = p
                vals[c] = 1.0
                c += 1
                continue

            # sumbu  $r = 0$  (regularitas  $dA/dr = 0$ )
            if i == 0:
                rows[c] = p
                cols[c] = p
                vals[c] = 1.0
                c += 1
                rows[c] = p
                cols[c] = p + Nz
                vals[c] = -1.0
                c += 1
                continue

            rp = r[i]
            ae = 1 / dr**2 + 1 / (2 * rp * dr)
            aw = 1 / dr**2 - 1 / (2 * rp * dr)
            an = 1 / dz**2
            ass = 1 / dz**2
            ap = -(ae + aw + an + ass) - 1 / rp**2          # KOREKSI: suku  $-A\phi_i/r^2$ 

            rows[c] = p
            cols[c] = (i + 1) * Nz + j
            vals[c] = ae
            c += 1

```

```

        rows[c] = p
        cols[c] = (i - 1) * Nz + j
        vals[c] = aw
        c += 1
        rows[c] = p
        cols[c] = p + 1
        vals[c] = an
        c += 1
        rows[c] = p
        cols[c] = p - 1
        vals[c] = ass
        c += 1
        rows[c] = p
        cols[c] = p
        vals[c] = ap
        c += 1
        rhs[p] = -mu0 * J[i, j]

    return rows[:c], cols[:c], vals[:c], rhs

# Selesaikan medan untuk satu posisi koil (pusat zc)
def solve(zc):
    J = np.zeros((Nr, Nz))
    za = zc - coil_h / 2
    zb = zc + coil_h / 2
    mask = (r[:, None] >= r1) & (r[:, None] <= r2) & (z[None, :] >= za) & (z[None, :] <=
        zb)
    J[mask] = J0

    rows, cols, vals, rhs = rakit(Nr, Nz, r, dr, dz, J, mu0)
    A = sp.csr_matrix((vals, (rows, cols)), shape=(Nr * Nz, Nr * Nz))
    Aphi = spla.spsolve(A, rhs).reshape((Nr, Nz))

    Br = np.zeros_like(Aphi)
    Bz = np.zeros_like(Aphi)
    Br[:, 1:-1] = -(Aphi[:, 2:] - Aphi[:, :-2]) / (2 * dz)
    for i in range(1, Nr - 1):
        # KOREKSI Bz = (1/r) d(r Aphi)/dr
        Bz[i, :] = (r[i + 1] * Aphi[i + 1, :] - r[i - 1] * Aphi[i - 1, :]) / (2 * dr * r
            [i])
    return np.sqrt(Br**2 + Bz**2), Br, Bz

# Sapuan posisi koil sepanjang sumbu (gerak autofokus)
zs = np.linspace(7e-3, 13e-3, 30)
FB = []
FR = []
FZ = []
print("Menyelesaikan medan magnet untuk tiap posisi koil ...")
for zc in zs:

```

```

    b, br, bz = solve(zc)
    FB.append(b)
    FR.append(br)
    FZ.append(bz)
print("Selesai.")

Bmx = max(b.max() for b in FB)
ic = np.argmin(np.abs(r - 2e-3))
ext = [0, Rmax * 1e3, 0, Zmax * 1e3]
Rg, Zg = np.meshgrid(r * 1e3, z * 1e3, indexing="ij")

# Vektor satuan supaya panah pendek dan hanya menunjukkan arah
def arah(U, V):
    M = np.hypot(U, V) + 1e-30
    return U / M, V / M

# Dashboard animasi: |B| + arah medan + profil sumbu
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(15, 4.6))
fig.suptitle("DISTRIBUSI MEDAN MAGNET AKTUATOR VCM (STROKE AUTOFOKUS)",
             fontsize=12, fontweight="bold", color="darkgreen")

im1 = ax1.imshow(FB[0].T, origin="lower", extent=ext, cmap="inferno", vmin=0, vmax=Bmx,
                 aspect="auto")
ax1.set_title("Distribusi Medan Magnet VCM")
ax1.set_xlabel("r (mm)")
ax1.set_ylabel("z (mm)")
fig.colorbar(im1, ax=ax1, label="|B| (Tesla)")

im2 = ax2.imshow(FB[0].T, origin="lower", extent=ext, cmap="viridis", vmin=0, vmax=Bmx,
                 aspect="auto")
u0, v0 = arah(FR[0][::5, ::5], FZ[0][::5, ::5])
qv = ax2.quiver(Rg[:, :5, ::5], Zg[:, :5, ::5], u0, v0, color="white", scale=30, width
               =0.004)
ax2.set_title("Arah Medan Magnet")
ax2.set_xlabel("r (mm)")
ax2.set_ylabel("z (mm)")

ln, = ax3.plot(z * 1e3, FB[0][ic, :], lw=2)
ax3.set_ylim(0, Bmx * 1.05)
ax3.set_title("Profil |B| Sepanjang Sumbu Aktuator")
ax3.set_xlabel("z (mm)")
ax3.set_ylabel("|B| (Tesla)")
ax3.grid(True)

def update(f):
    im1.set_data(FB[f].T)
    im2.set_data(FB[f].T)
    qv.set_UVC(*arah(FR[f][::5, ::5], FZ[f][::5, ::5]))

```

```

    ln.set_ydata(FB[f][ic, :])
    ax1.set_title("Distribusi Medan Magnet VCM (koil z = %.1f mm)" % (zs[f] * 1e3))
    return [im1, im2, qv, ln]

ani = FuncAnimation(fig, update, frames=len(FB), interval=120, blit=False)
plt.tight_layout()
plt.close()

# Plot statis (posisi koil di tengah)
fmid = len(FB) // 2
figs, (b1, b2, b3) = plt.subplots(1, 3, figsize=(15, 4.6))
figs.suptitle("DISTRIBUSI MEDAN MAGNET VCM (koil z = %.1f mm)" % (zs[fmid] * 1e3),
             fontweight="bold")

s1 = b1.imshow(FB[fmid].T, origin="lower", extent=ext, cmap="inferno", vmin=0, vmax=Bmx,
              aspect="auto")
b1.set_title("Distribusi Medan Magnet VCM")
b1.set_xlabel("r (mm)")
b1.set_ylabel("z (mm)")
figs.colorbar(s1, ax=b1, label="|B| (Tesla)")

b2.imshow(FB[fmid].T, origin="lower", extent=ext, cmap="viridis", vmin=0, vmax=Bmx,
          aspect="auto")
us, vs = arah(FR[fmid][:5, :5], FZ[fmid][:5, :5])
b2.quiver(Rg[:5, :5], Zg[:5, :5], us, vs, color="white", scale=30, width=0.004)
b2.set_title("Arah Medan Magnet")
b2.set_xlabel("r (mm)")
b2.set_ylabel("z (mm)")

b3.plot(z * 1e3, FB[fmid][ic, :], lw=2)
b3.set_title("Profil |B| Sepanjang Sumbu Aktuator")
b3.set_xlabel("z (mm)")
b3.set_ylabel("|B| (Tesla)")
b3.grid(True)

figs.tight_layout()

# Output: animasi video + plot statis
# display(HTML(ani.to_html5_video()))
figs.savefig('../figures/kasus4.png', dpi=200, bbox_inches='tight')
# plt.show()

```

Listing Program Kasus 5

```

# Kasus 5 - Distribusi medan magnet OIS (bola termagnetisasi seragam)

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation

```

```

from IPython.display import HTML, display

# Parameter magnet
mu0 = 4 * np.pi * 1e-7
R = 1.0
M = 1.0e6 # magnetisasi (A/m), orde magnet NdFeB
m = (4 / 3) * np.pi * R**3 * M # momen dipol magnetik

# Domain
x = np.linspace(-4, 4, 300)
y = np.linspace(-4, 4, 300)
X, Y = np.meshgrid(x, y)
r = np.sqrt(X**2 + Y**2)
ca = X / (r + 1e-12)
sa = Y / (r + 1e-12)
ins = r < R
mid = len(y) // 2

# Medan untuk arah magnetisasi phi
def field(phi):
    mx = np.cos(phi)
    my = np.sin(phi)
    mdotr = mx * ca + my * sa
    pref = (mu0 / (4 * np.pi)) * m / (r**3 + 1e-12)
    Bx = pref * (3 * mdotr * ca - mx) # ekterior: dipol
    By = pref * (3 * mdotr * sa - my)
    Bx[ins] = (2 / 3) * mu0 * M * mx # interior: seragam (2/3) mu0 M
    By[ins] = (2 / 3) * mu0 * M * my
    return Bx, By

phis = np.linspace(0, 2 * np.pi, 30, endpoint=False)
Bx0, By0 = field(0.0)
Bmag0 = np.sqrt(Bx0**2 + By0**2)
vmax = np.percentile(Bmag0, 99)
print("B interior =", (2 / 3) * mu0 * M, "Tesla")

# Dashboard animasi: |B| + garis fluks + profil
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(15, 5))
fig.suptitle("DISTRIBUSI MEDAN MAGNET OIS (BOLA TERMAGNETISASI)",
             fontsize=12, fontweight="bold", color="purple")

im = ax1.imshow(Bmag0, origin="lower", extent=[-4, 4, -4, 4], cmap="inferno", vmin=0,
                vmax=vmax)
ax1.set_title("Distribusi Medan Magnet")
ax1.set_xlabel("x")
ax1.set_ylabel("y")
fig.colorbar(im, ax=ax1, label="|B| (Tesla)")

ax2.streamplot(X, Y, Bx0, By0, density=1.4)

```

```

ax2.set_title("Garis Fluks Magnetik")
ax2.set_xlabel("x")
ax2.set_ylabel("y")

ln, = ax3.plot(x, Bmag0[mid, :], lw=2)
ax3.set_ylim(0, vmax * 1.1)
ax3.set_title("Profil Medan Magnet")
ax3.set_xlabel("x")
ax3.set_ylabel("|B| (Tesla)")
ax3.grid(True)

def update(f):
    Bx, By = field(phis[f])
    Bm = np.sqrt(Bx**2 + By**2)
    im.set_data(Bm)
    ax2.clear()
    ax2.set_title("Garis Fluks Magnetik")
    ax2.set_xlabel("x")
    ax2.set_ylabel("y")
    ax2.streamplot(X, Y, Bx, By, density=1.4)
    ln.set_ydata(Bm[mid, :])
    ax1.set_title("Distribusi Medan Magnet (arah %d deg)" % int(np.degrees(phis[f])))
    return [im, ln]

ani = FuncAnimation(fig, update, frames=len(phis), interval=120, blit=False)
plt.tight_layout()
plt.close()

# Plot statis (arah magnetisasi sumbu x)
figs, (b1, b2, b3) = plt.subplots(1, 3, figsize=(15, 5))
figs.suptitle("DISTRIBUSI MEDAN MAGNET OIS (arah magnetisasi sumbu x)", fontweight="bold")

s1 = b1.imshow(Bmag0, origin="lower", extent=[-4, 4, -4, 4], cmap="inferno", vmin=0,
               vmax=vmax)
b1.set_title("Distribusi Medan Magnet")
b1.set_xlabel("x")
b1.set_ylabel("y")
figs.colorbar(s1, ax=b1, label="|B| (Tesla)")

b2.streamplot(X, Y, Bx0, By0, density=1.4)
b2.set_title("Garis Fluks Magnetik")
b2.set_xlabel("x")
b2.set_ylabel("y")

b3.plot(x, Bmag0[mid, :], lw=2)
b3.set_title("Profil Medan Magnet")
b3.set_xlabel("x")
b3.set_ylabel("|B| (Tesla)")

```

```

b3.grid(True)

figs.tight_layout()

# Output: animasi video + plot statis
# display(HTML(ani.to_html5_video()))
figs.savefig('../figures/kasus5.png', dpi=200, bbox_inches='tight')
# plt.show()

```

Listing Program Kasus 6

```

# Kasus 6 - Propagasi cahaya dome port + lensa menuju sensor CMOS

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from IPython.display import HTML, display
from numba import njit, prange

# Konstanta fisis
c0 = 299792458.0
eps0 = 8.8541878128e-12
mu0 = 4e-7 * np.pi
lam = 550e-9
omega = 2 * np.pi * c0 / lam

# Grid (konvensi Kakai: i = Grid X, j = arah rambat = Grid Y, k = kedalaman)
dx = dy = dz = 25e-9
NX, NY, NK = 200, 200, 80
kmid = NK // 2
dt = 0.99 / (c0 * np.sqrt(1 / dx**2 + 1 / dy**2 + 1 / dz**2)) # syarat CFL 3D
nstep = 800 # iterasi penuh agar
    citra terbentuk jelas
simpan = 6

# Bidang penting
jsrc = 15 # bidang injeksi gelombang
j_sensor = 185 # bidang sensor CMOS (bidang fokus, dari pemindaian)
acc_start = 300 # mulai akumulasi citra setelah gelombang mencapai sensor

# Peta indeks bias: geometri 2D Kakai diekstrusi jadi 3D (revolusi terhadap sumbu optik)
i = np.arange(NX)[: , None, None]
j = np.arange(NY)[None, :, None]
k = np.arange(NK)[None, None, :]
center_x, center_y = 100, 170
r_luar, r_dalam = 135, 120

n = np.ones((NX, NY, NK)) * 1.33 # air (n = 1.33)
rr = np.sqrt((i - center_x)**2 + (j - center_y)**2 + (k - kmid)**2)
n[(rr >= r_dalam) & (rr <= r_luar)] = 1.52 # cangkang dome (kaca BK7)

```



```

n[rr < r_dalam] = 1.00 # udara dalam kamera
lensa = ((i - 100) / 40.0)**2 + ((k - kmid) / 40.0)**2 + ((j - 140) / 18.0)**2
n[lensa <= 1] = 1.50 # lensa biconvex (kaca)

De = dt / (eps0 * n**2)
Dh = dt / mu0
tap = np.hanning(NX)[:, None] * np.hanning(NK)[None, :]

# Kernel Yee 3D (Numba parallel)
@njit(parallel=True, fastmath=True)
def upd_H(Ex, Ey, Ez, Hx, Hy, Hz, Dh, dx, dy, dz):
    NX, NY, NK = Ex.shape
    for a in prange(NX):
        for b in range(NY - 1):
            for c in range(NK - 1):
                Hx[a, b, c] += Dh * ((Ey[a, b, c + 1] - Ey[a, b, c]) / dz - (Ez[a, b + 1, c] - Ez[a, b, c]) / dy)
    for a in prange(NX - 1):
        for b in range(NY):
            for c in range(NK - 1):
                Hy[a, b, c] += Dh * ((Ez[a + 1, b, c] - Ez[a, b, c]) / dx - (Ex[a, b, c + 1] - Ex[a, b, c]) / dz)
    for a in prange(NX - 1):
        for b in range(NY - 1):
            for c in range(NK):
                Hz[a, b, c] += Dh * ((Ex[a, b + 1, c] - Ex[a, b, c]) / dy - (Ey[a + 1, b, c] - Ey[a, b, c]) / dx)

@njit(parallel=True, fastmath=True)
def upd_E(Ex, Ey, Ez, Hx, Hy, Hz, De, dx, dy, dz):
    NX, NY, NK = Ex.shape
    for a in prange(NX):
        for b in range(1, NY):
            for c in range(1, NK):
                Ex[a, b, c] += De[a, b, c] * ((Hz[a, b, c] - Hz[a, b - 1, c]) / dy - (Hy[a, b, c] - Hy[a, b, c - 1]) / dz)
    for a in prange(1, NX):
        for b in range(NY):
            for c in range(1, NK):
                Ey[a, b, c] += De[a, b, c] * ((Hx[a, b, c] - Hx[a, b, c - 1]) / dz - (Hz[a, b, c] - Hz[a - 1, b, c]) / dx)
    for a in prange(1, NX):
        for b in range(1, NY):
            for c in range(NK):
                Ez[a, b, c] += De[a, b, c] * ((Hy[a, b, c] - Hy[a - 1, b, c]) / dx - (Hx[a, b, c] - Hx[a, b - 1, c]) / dy)

# Inisialisasi medan (sumber terpolarisasi Ez, seperti mode TMz Kakai)

```

```

Ex = np.zeros((NX, NY, NK))
Ey = np.zeros((NX, NY, NK))
Ez = np.zeros((NX, NY, NK))
Hx = np.zeros((NX, NY, NK))
Hy = np.zeros((NX, NY, NK))
Hz = np.zeros((NX, NY, NK))
acc = np.zeros((NX, NK))

data_Ez = []
data_prof = []
data_cit = []
data_t = []

print("Menjalankan FDTD Yee 3D ...")
for s in range(nstep):
    upd_H(Ex, Ey, Ez, Hx, Hy, Hz, Dh, dx, dy, dz)
    upd_E(Ex, Ey, Ez, Hx, Hy, Hz, De, dx, dy, dz)

    # Sumber gelombang bidang
    Ez[:, jsrc, :] += np.sin(omega * s * dt) * tap * 0.5

    # Batas serap sederhana (ABC) di tepi domain
    ab = 0.98
    for F in (Ex, Ey, Ez):
        F[:, 0, :] *= ab
        F[:, -1, :] *= ab
        F[0, :, :] *= ab
        F[-1, :, :] *= ab
        F[:, :, 0] *= ab
        F[:, :, -1] *= ab

    # Akumulasi intensitas pada bidang sensor untuk membentuk citra
    if s > acc_start:
        acc += Ex[:, j_sensor, :]**2 + Ey[:, j_sensor, :]**2 + Ez[:, j_sensor, :]**2

    # Simpan frame
    if s % simpan == 0:
        data_Ez.append(Ez[:, :, kmid].copy())
        prof = Ez[:, j_sensor, kmid]**2 + Ex[:, j_sensor, kmid]**2 + Ey[:, j_sensor,
        kmid]**2
        data_prof.append(prof / (prof.max() + 1e-12))
        data_cit.append(acc.copy())
        data_t.append(s * dt * 1e15)

print("Selesai.")

nsl = n[:, :, kmid]
cropi = slice(60, 140) # crop citra jadi kotak di sekitar fokus
cit_max = data_cit[-1][cropi, :].max() + 1e-12 # skala tetap agar citra terlihat
tumbuh

```

```

# Dashboard animasi: peta Ez (Kakai) + profil intensitas (Kakai) + citra sensor (baru)
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(15, 4.6))
fig.suptitle("PROPAGASI GELOMBANG CAHAYA MELALUI DOME PORT DAN LENSE (FDTD YEE 3D)",
             fontsize=11, fontweight="bold", color="darkblue")

im = ax1.imshow(data_Ez[0].T, cmap="RdBu", origin="lower", vmin=-0.2, vmax=0.2)
ax1.contour(nsl.T, levels=[1.1, 1.4], colors=["white", "cyan"], alpha=0.5, linestyle="--",
            linewidths=0.9)
ax1.axhline(j_sensor, color="lime", ls=":", label="Sensor CMOS")
ax1.set_title("Distribusi Medan Listrik (Ez)")
ax1.set_xlabel("Grid X")
ax1.set_ylabel("Grid Y")
ax1.legend(loc="upper right", fontsize=8)
fig.colorbar(im, ax=ax1, label="Amplitudo Ez")

line, = ax2.plot(np.arange(NX), data_prof[0], "g-", lw=2)
ax2.set_title("Profil Intensitas pada Sensor CMOS")
ax2.set_xlabel("Posisi Piksel Sensor")
ax2.set_ylabel("Intensitas Relatif")
ax2.set_xlim(0, NX)
ax2.set_ylim(0, 1.05)
ax2.grid(alpha=0.4)

ic = ax3.imshow(data_cit[0][cropi, :].T, cmap="inferno", origin="lower",
                interpolation="bilinear", aspect="auto", vmin=0, vmax=cit_max)
ax3.set_title("Citra Terbentuk di Sensor CMOS")
ax3.set_xlabel("Piksel X")
ax3.set_ylabel("Piksel Y")
fig.colorbar(ic, ax=ax3, label="Intensitas terkumpul (a.u.)")

def update(f):
    im.set_data(data_Ez[f].T)
    line.set_ydata(data_prof[f])
    ic.set_data(data_cit[f][cropi, :].T)
    ax1.set_xlabel("Grid X | t = %.1f fs" % data_t[f])
    return [im, line, ic]

ani = FuncAnimation(fig, update, frames=len(data_Ez), interval=80, blit=False)
plt.tight_layout()
plt.close()

# Plot statis kondisi akhir (2x2 Grid dengan 3D Surface)
figs = plt.figure(figsize=(12, 10))
figs.suptitle("PROPAGASI GELOMBANG CAHAYA MENUJU SENSOR CMOS (FDTD YEE 3D)", fontweight="bold",
             fontsize=14)

# Panel 1: 3D Surface Plot
b0 = figs.add_subplot(2, 2, 1, projection='3d')
X_mesh, Y_mesh = np.meshgrid(np.arange(NX), np.arange(NY), indexing='ij')

```

```

surf = b0.plot_surface(X_mesh, Y_mesh, data_Ez[-1], cmap="RdBu", rstride=2, cstride=2,
                      alpha=0.9, antialiased=True, vmin=-0.2, vmax=0.2)
b0.set_title("Proyeksi Permukaan 3D Medan Listrik (Ez)")
b0.set_xlabel("Grid X")
b0.set_ylabel("Grid Y")
b0.set_zlabel("Amplitudo Ez")
b0.view_init(elev=45, azim=-60)
figs.colorbar(surf, ax=b0, shrink=0.5, pad=0.1, label="Ez")

# Panel 2: Peta 2D (sebelumnya b1)
b1 = figs.add_subplot(2, 2, 2)
p1 = b1.imshow(data_Ez[-1].T, cmap="RdBu", origin="lower", vmin=-0.2, vmax=0.2)
b1.contour(nsl.T, levels=[1.1, 1.4], colors=["white", "cyan"], alpha=0.5, linestyle="--",
           linewidths=0.9)
b1.axhline(j_sensor, color="lime", ls=":")
b1.set_title("Peta 2D Distribusi Medan Listrik (Ez)")
b1.set_xlabel("Grid X")
b1.set_ylabel("Grid Y")
figs.colorbar(p1, ax=b1, label="Ez")

# Panel 3: Profil Intensitas
b2 = figs.add_subplot(2, 2, 3)
b2.plot(np.arange(NX), data_prof[-1], "g-", lw=2)
b2.set_title("Profil Intensitas pada Sensor CMOS")
b2.set_xlabel("Posisi Piksel Sensor")
b2.set_ylabel("Intensitas Relatif")
b2.grid(alpha=0.4)

# Panel 4: Citra
b3 = figs.add_subplot(2, 2, 4)
icf = b3.imshow(acc[cropi, :].T, cmap="inferno", origin="lower", interpolation="bilinear",
                aspect="auto")
b3.set_title("Citra di Sensor CMOS")
b3.set_xlabel("Piksel X")
b3.set_ylabel("Piksel Y")
figs.colorbar(icf, ax=b3, label="Intensitas (a.u.)")

figs.tight_layout()

# Output: animasi video + plot statis
# display(HTML(ani.to_html5_video()))
figs.savefig('../figures/kasus6.png', dpi=200, bbox_inches='tight')
# plt.show()

```

Listing Program Kasus 7

```

# Kasus 7 - Schrodinger 2D TDSE Crank-Nicolson, citra dari probabilitas
import numpy as np
import scipy.sparse as sp
import scipy.sparse.linalg as spla

```

```

import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from IPython.display import HTML, display
from numba import njit, prange

Lx, Ly = 60.0, 40.0
Nx, Ny = 200, 130
dx, dy = Lx / Nx, Ly / Ny
x = (np.arange(Nx) + 0.5) * dx - Lx / 2
y = (np.arange(Ny) + 0.5) * dy - Ly / 2
X, Y = np.meshgrid(x, y, indexing="ij")
dt = 0.02
nstep = 1000

# potensial: sumur deplesi + penghalang STI dengan celah
U = np.zeros((Nx, Ny))
U[(X > 8) & (X < 22) & (np.abs(Y) < 10)] = -1.2
U[(np.abs(X - 2.0) < 0.8) & ~(np.abs(Y) < 2.2)] = 4.0

# CAP penyerap tepi
cap = np.zeros((Nx, Ny))
for arr, L in ((np.abs(X), Lx / 2), (np.abs(Y), Ly / 2)):
    s = np.clip((arr - (L - 5.0)) / 5.0, 0, 1)
    cap += 3.0 * s**2
Uc = U - 1j * cap

# Hamiltonian 5 titik dan operator Crank-Nicolson
N = Nx * Ny
Dx = sp.diags([1.0, -2.0, 1.0], [-1, 0, 1], shape=(Nx, Nx)) / dx**2
Dy = sp.diags([1.0, -2.0, 1.0], [-1, 0, 1], shape=(Ny, Ny)) / dy**2
H = -0.5 * (sp.kron(Dx, sp.identity(Ny)) + sp.kron(sp.identity(Nx), Dy)) + sp.diags(Uc,
    ravel())
A_cn = (sp.identity(N, dtype=complex) + 0.5j * dt * H).tocsc()
B_cn = (sp.identity(N, dtype=complex) - 0.5j * dt * H).tocsc()
lu = spla.splu(A_cn)

# paket gelombang elektron foto-eksitasi
x0, y0, sigma, k0 = -18.0, 0.0, 3.0, 2.2
psi = np.exp(-((X - x0)**2 + (Y - y0)**2) / (2 * sigma**2)) * np.exp(1j * k0 * X)
psi = (psi / np.sqrt(np.sum(np.abs(psi)**2) * dx * dy)).ravel().astype(complex)
well = ((X > 8) & (X < 22) & (np.abs(Y) < 10)).ravel()

@njit(parallel=True)
def akumulasi(citra, prob, dt):
    n = prob.shape[0]
    for p in prange(n):
        citra[p] += prob[p] * dt

# integrasi waktu

```

```

citra = np.zeros(N)                                # peta muatan terkumpul (dari probabilitas)
frames_psi, frames_citra = [], []
pw, ts = [], []
for n in range(nstep + 1):
    prob = np.abs(psi)**2
    akumulasi(citra, prob, dt)
    pw.append(float(np.sum(prob[well]) * dx * dy))
    ts.append(n * dt)
    if n % 20 == 0:
        frames_psi.append(prob.reshape(Nx, Ny).T.copy())
        frames_citra.append(citra.reshape(Nx, Ny).T.copy())
    if n < nstep:
        psi = lu.solve(B_cn @ psi)

print("peluang tangkap puncak =", round(max(pw) * 100, 1), "% pada t =", round(ts[int(
    np.argmax(pw))], 1))
print("peluang tangkap akhir =", round(pw[-1] * 100, 1), "%")

fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(15, 4.6), constrained_layout=True)
fig.suptitle("DINAMIKA ELEKTRON FOTO-EKSITASI PADA SENSOR CMOS (SCHRODINGER 2D)",
    fontsize=11, fontweight="bold")

im1 = ax1.imshow(frames_psi[0], origin="lower", cmap="magma",
    extent=[x[0], x[-1], y[0], y[-1]], aspect="auto", interpolation="
    bilinear")
ax1.contour(x, y, (U.T > 2), levels=[0.5], colors="cyan", linewidths=1.2)
ax1.contour(x, y, (U.T < -0.5), levels=[0.5], colors="lime", linewidths=1.2)
ax1.set_title("Rapat peluang  $|\psi|^2$  (elektron bergerak)")
ax1.set_xlabel("x")
ax1.set_ylabel("y")

im2 = ax2.imshow(frames_citra[0], origin="lower", cmap="cividis",
    extent=[x[0], x[-1], y[0], y[-1]], aspect="auto", interpolation="
    bilinear")
fig.colorbar(im2, ax=ax2, label="muatan terkumpul (a.u.)")
ax2.contour(x, y, (U.T < -0.5), levels=[0.5], colors="lime", linewidths=1.2)
ax2.set_title("Citra dari probabilitas (sinyal piksel)")
ax2.set_xlabel("x")
ax2.set_ylabel("y")

garis, = ax3.plot([], [], "g-", lw=2, label="P(dalam sumur)")
ax3.set_xlim(0, nstep * dt)
ax3.set_ylim(0, 1)
ax3.grid(alpha=0.3)
ax3.set_title("Efisiensi penangkapan elektron")
ax3.set_xlabel("t")
ax3.set_ylabel("probabilitas")
ax3.legend(loc="upper left", fontsize=8)

ft = ts[::20]
fp = pw[::20]

```

```

def update(f):
    im1.set_data(frames_psi[f])
    im1.set_clim(0, max(frames_psi[f].max(), 1e-9))
    im2.set_data(frames_citra[f])
    im2.set_clim(0, max(frames_citra[f].max(), 1e-9))
    garis.set_data(ft[:f + 1], fp[:f + 1])
    return im1, im2, garis

ani = FuncAnimation(fig, update, frames=len(frames_psi), interval=100, blit=False)
plt.close(fig)

# Plot statis kondisi akhir
figs, (cx1, cx2, cx3) = plt.subplots(1, 3, figsize=(15, 4.6), constrained_layout=True)
figs.suptitle("DINAMIKA ELEKTRON FOTO-EKSITASI PADA SENSOR CMOS (KONDISI AKHIR)",
              fontweight="bold")
cx1.imshow(frames_psi[-1], origin="lower", cmap="magma", extent=[x[0], x[-1], y[0], y
[-1]],
           aspect="auto", interpolation="bilinear")
cx1.contour(x, y, (U.T > 2), levels=[0.5], colors="cyan", linewidths=1.2)
cx1.contour(x, y, (U.T < -0.5), levels=[0.5], colors="lime", linewidths=1.2)
cx1.set_title("Rapat peluang akhir")
cx1.set_xlabel("x")
cx1.set_ylabel("y")
sm = cx2.imshow(frames_citra[-1], origin="lower", cmap="cividis", extent=[x[0], x[-1], y
[0], y[-1]],
               aspect="auto", interpolation="bilinear")
figs.colorbar(sm, ax=cx2, label="muatan terkumpul (a.u.)")
cx2.contour(x, y, (U.T < -0.5), levels=[0.5], colors="lime", linewidths=1.2)
cx2.set_title("Citra dari probabilitas (sinyal piksel)")
cx2.set_xlabel("x")
cx2.set_ylabel("y")
cx3.plot(ft, fp, "g-", lw=2)
cx3.set_xlim(0, nstep * dt)
cx3.set_ylim(0, 1)
cx3.grid(alpha=0.3)
cx3.set_title("Efisiensi penangkapan elektron")
cx3.set_xlabel("t")
cx3.set_ylabel("probabilitas")

# Output: animasi video + plot statis
# display(HTML(ani.to_html5_video()))
figs.savefig('../figures/kasus7.png', dpi=200, bbox_inches='tight')
# plt.show()

```